Hybrid Computing Technology

OpenCL Programming

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

FACULTY OF INFORMATION TECHNOLOGY

KMITL

Topics

- OpenCL Introduction
- OpenCL Programs and Execution Model
- OpenCL Context and Memory Model
- OpenCL Software
- OpenCL Example
- OpenCL versus CUDA
- Conclusion

What is OpenCL?

- Open Computing Language
  - standard for parallel programming of heterogeneous systems consisting of parallel processors like CPUs and GPUs
  - specification developed by many companies
  - maintained by the Khronos Group
    - OpenGL and other open spec. technologies
- Implemented by hardware vendors
  - implementation is compliant if it conforms to the specifications

What is an OpenCL device?

- Any piece of hardware that is OpenCL compliant
  - device
    - compute units
      - processing elements

multicore CPU

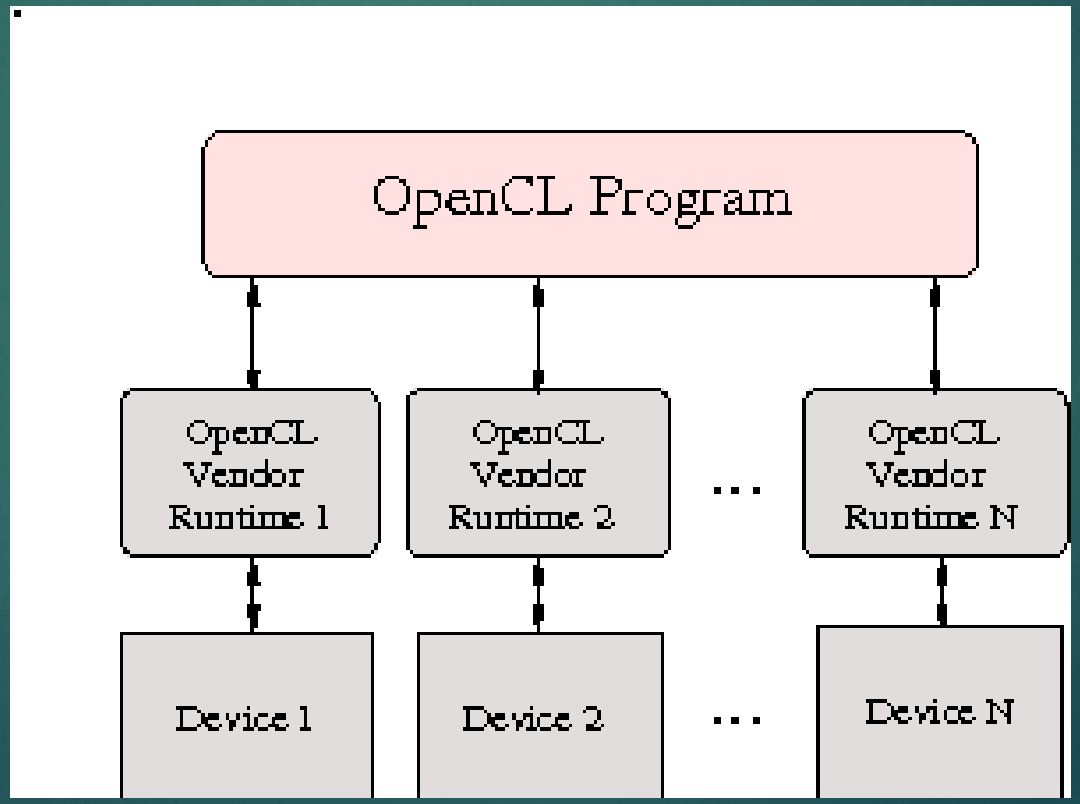
many graphics adapters
Nvidia
AMD

CUDA – Recent OpenCL Predecessor

- "Compute Unified Device Architecture"
- General purpose programming model
  - User kicks off batches of threads on the GPU
  - GPU = dedicated super-threaded, massively data parallel co-processor
- Targeted software stack
  - Compute oriented drivers, language, and tools
- Driver for loading computation programs into GPU
  - Standalone Driver - Optimized for computation
  - Interface designed for compute – graphics-free API
  - Data sharing with OpenGL buffer objects
  - Guaranteed maximum download & readback speeds
  - Explicit GPU memory management

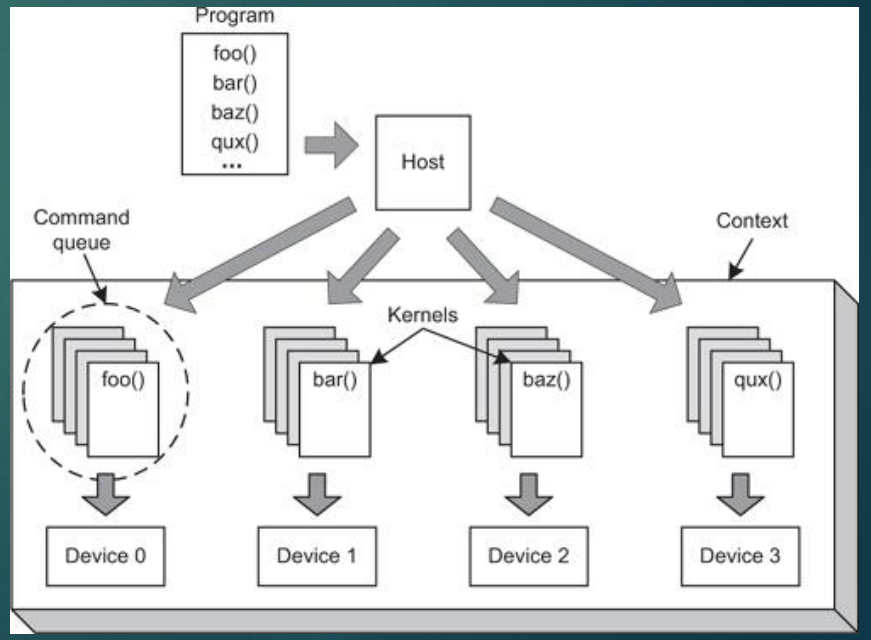
Challenges

- Vendor specific APIs
- CPU – GPGPU Programming gap



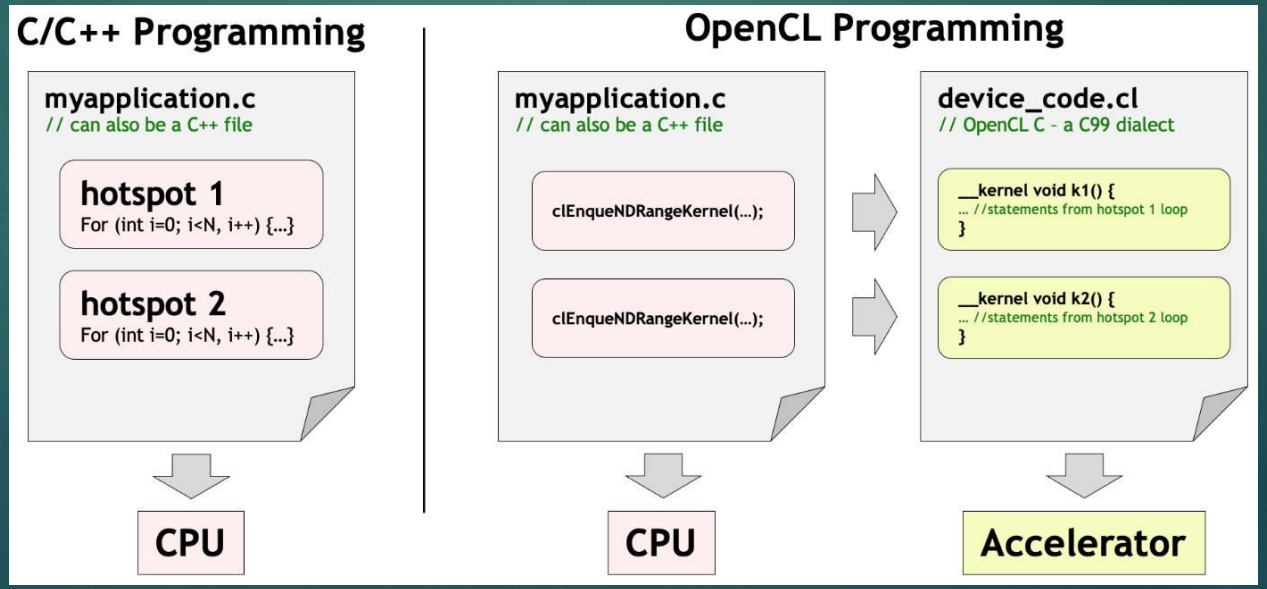
OpenCL

- Open Computing Language
- Introduces uniformity
- “Close-to-silicon”
- Parallel Computing using all possible resources on end system
- Initially by Apple
- Khronos group, OpenGL, OpenAL
- Major Vendor support



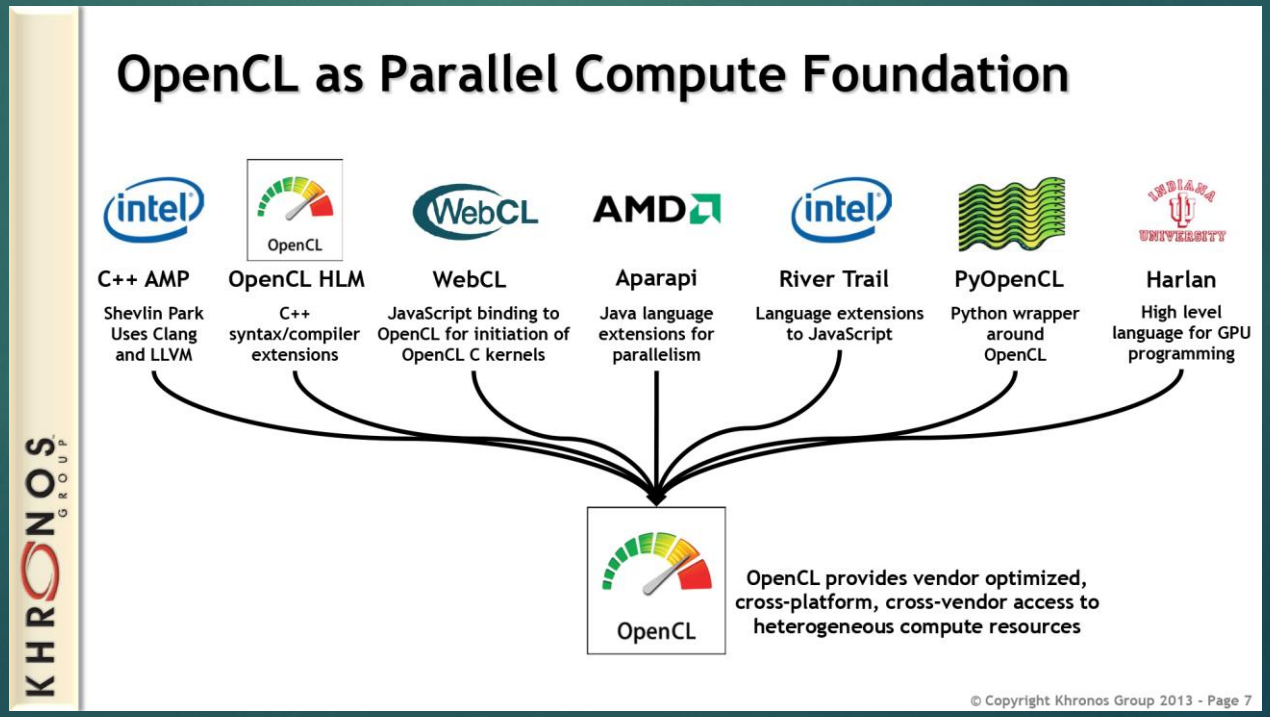
What is OpenCL?

- Cross-platform parallel computing API and C-like language for heterogeneous computing devices
- Code is portable across various target devices:
  - Correctness is guaranteed
  - Performance of a given kernel is not guaranteed across differing target devices
- OpenCL implementations already exist for AMD and NVIDIA GPUs, x86 CPUs
- In principle, OpenCL could also target DSPs, Cell, and perhaps also FPGAs

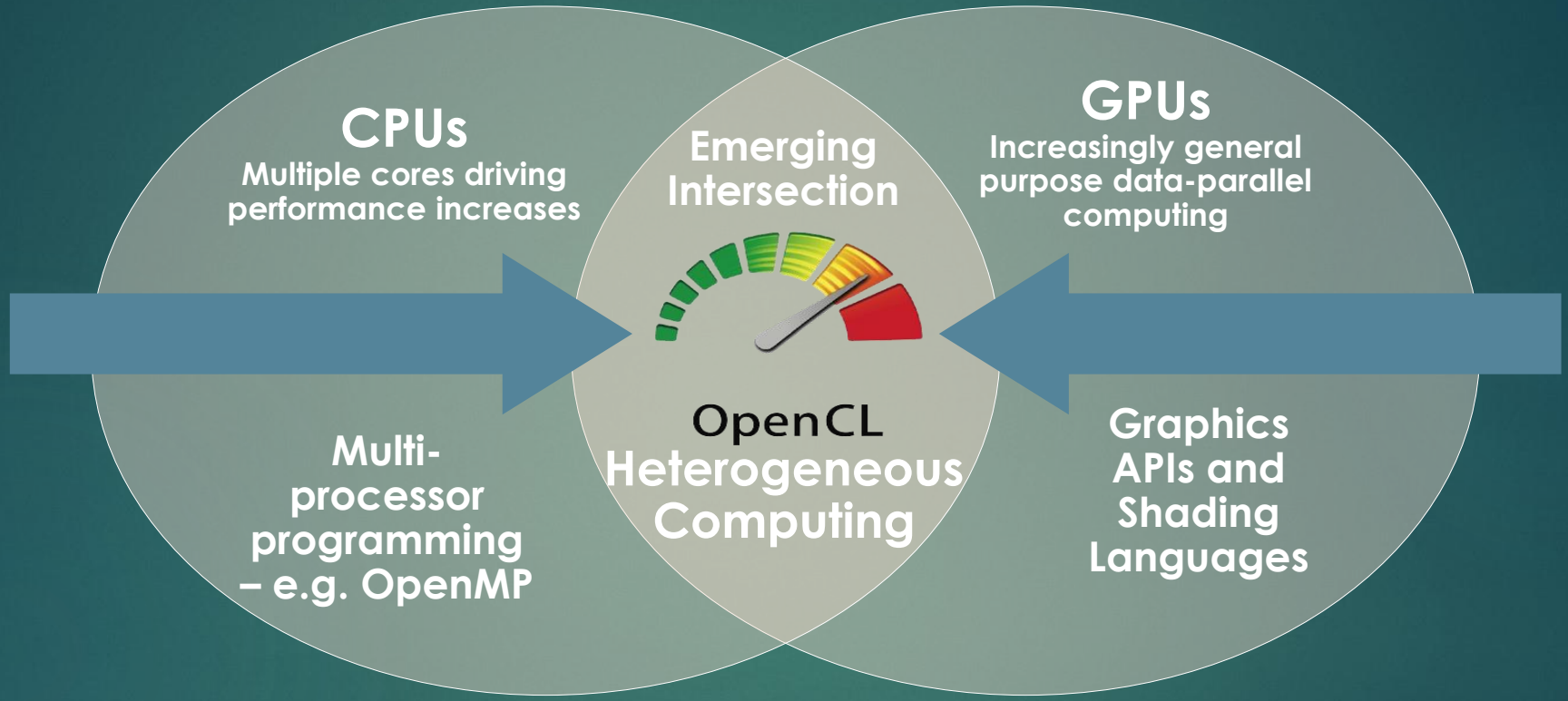


More on Multi-Platform Targeting

- Targets a broader range of CPU-like and GPU-like devices than CUDA
  - Targets devices produced by multiple vendors
  - Many features of OpenCL are optional and may not be supported on all devices
- OpenCL codes must be prepared to deal with much greater hardware diversity
- A single OpenCL kernel will likely not achieve peak performance on all device types



Industry Standards for Programming Heterogeneous Platforms



OpenCL – Open Computing Language

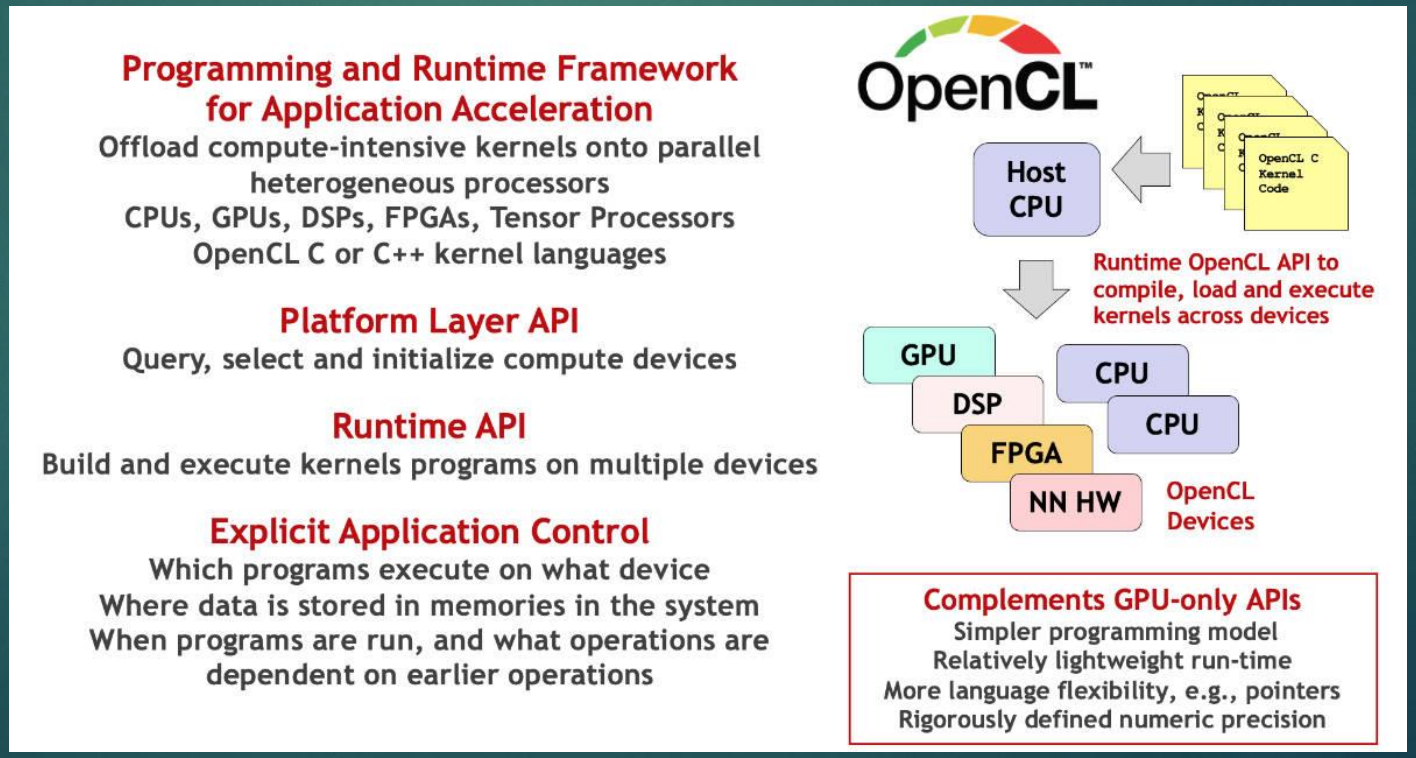
Open, royalty-free standard for portable, parallel programming of heterogeneous parallel computing CPUs, GPUs, and other processors

OpenCL Working Group within Khronos

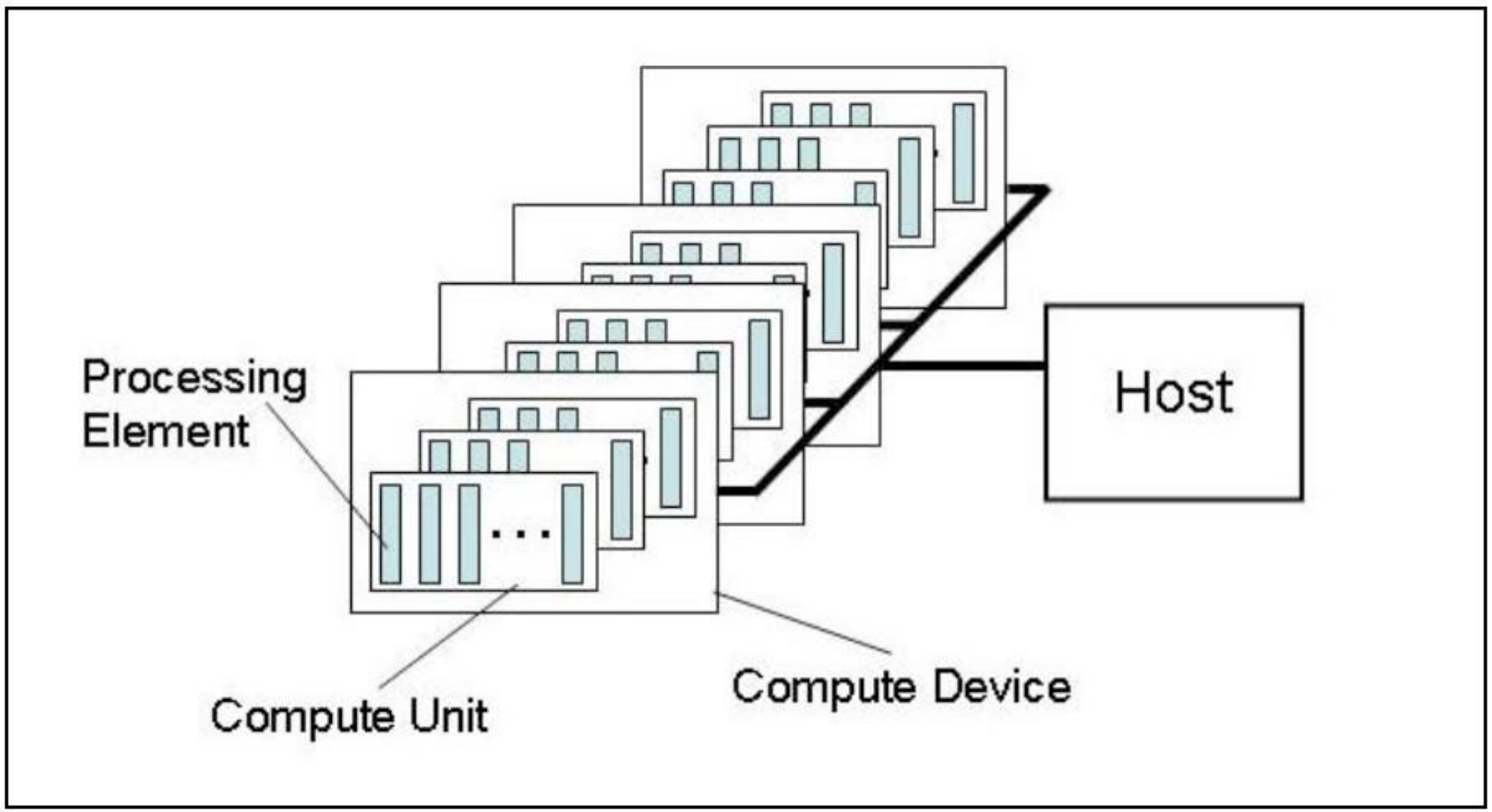
- Diverse industry participation
  - Processor vendors, system OEMs, middleware vendors, application developers.
- OpenCL became an important standard upon release by virtue of the market coverage of the companies behind it.

OpenCL Overview

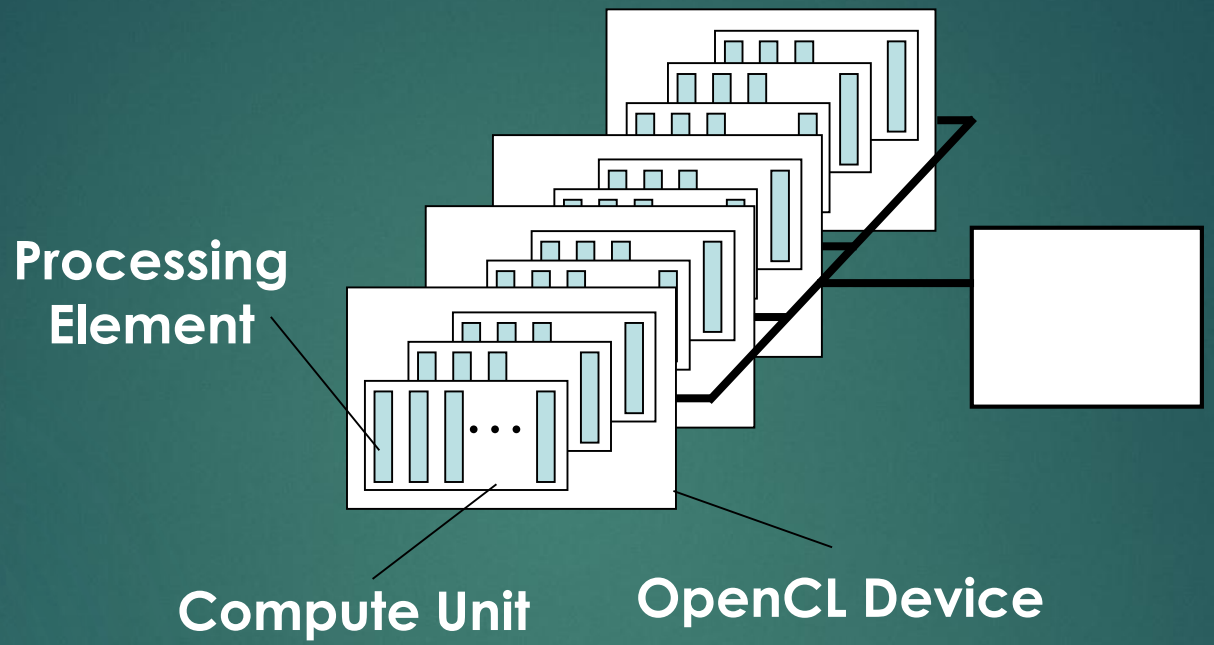
- All computational resources on an end system seen as peers
- CPU, GPU, ARM, DSPs etc
- Strict IEEE 754 Floating Point specification. Fixed rounding, error
- Defines architecture models and software stack



Architecture Model – Platform



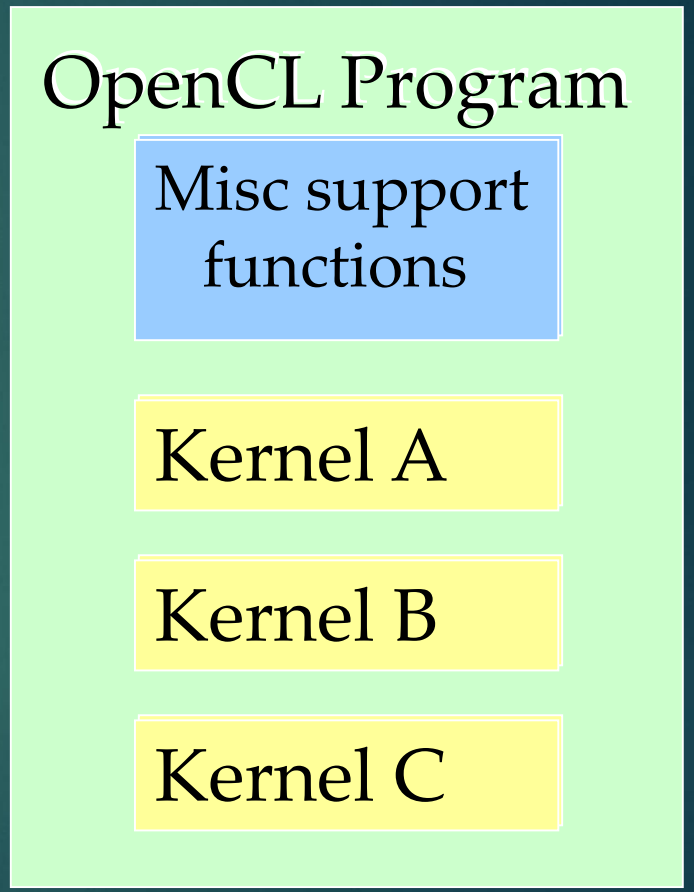
OpenCL Platform Model



- One ***Host*** and one or more ***OpenCL Devices***
  - Each OpenCL Device is composed of one or more ***Compute Units***
    - Each Compute Unit is divided into one or more ***Processing Elements***
- Memory divided into ***host memory*** and ***device memory***

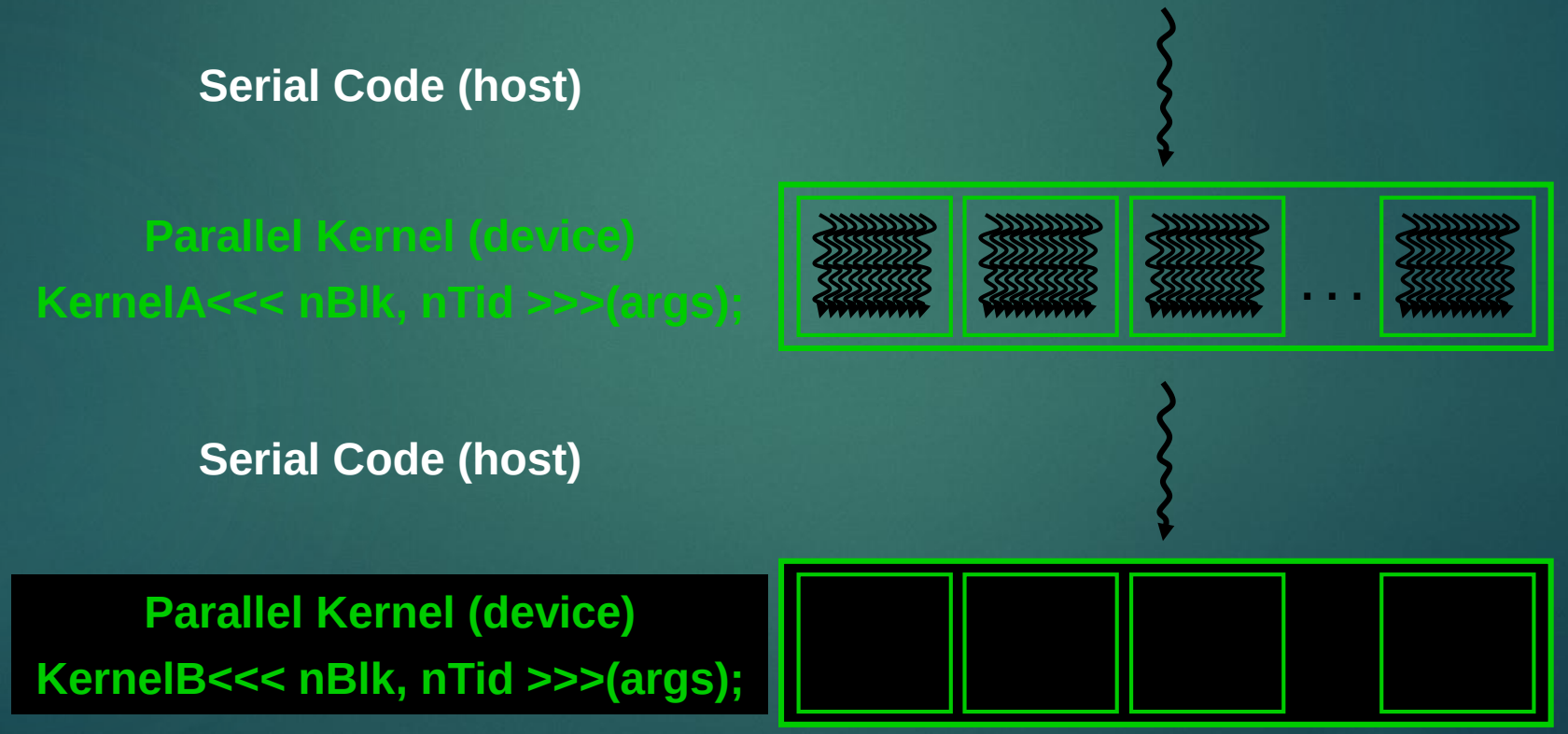
OpenCL Programs

- An OpenCL "program" contains one or more "kernels" and any supporting routines that run on a target device
- An OpenCL kernel is the basic unit of parallel code that can be executed on a target device



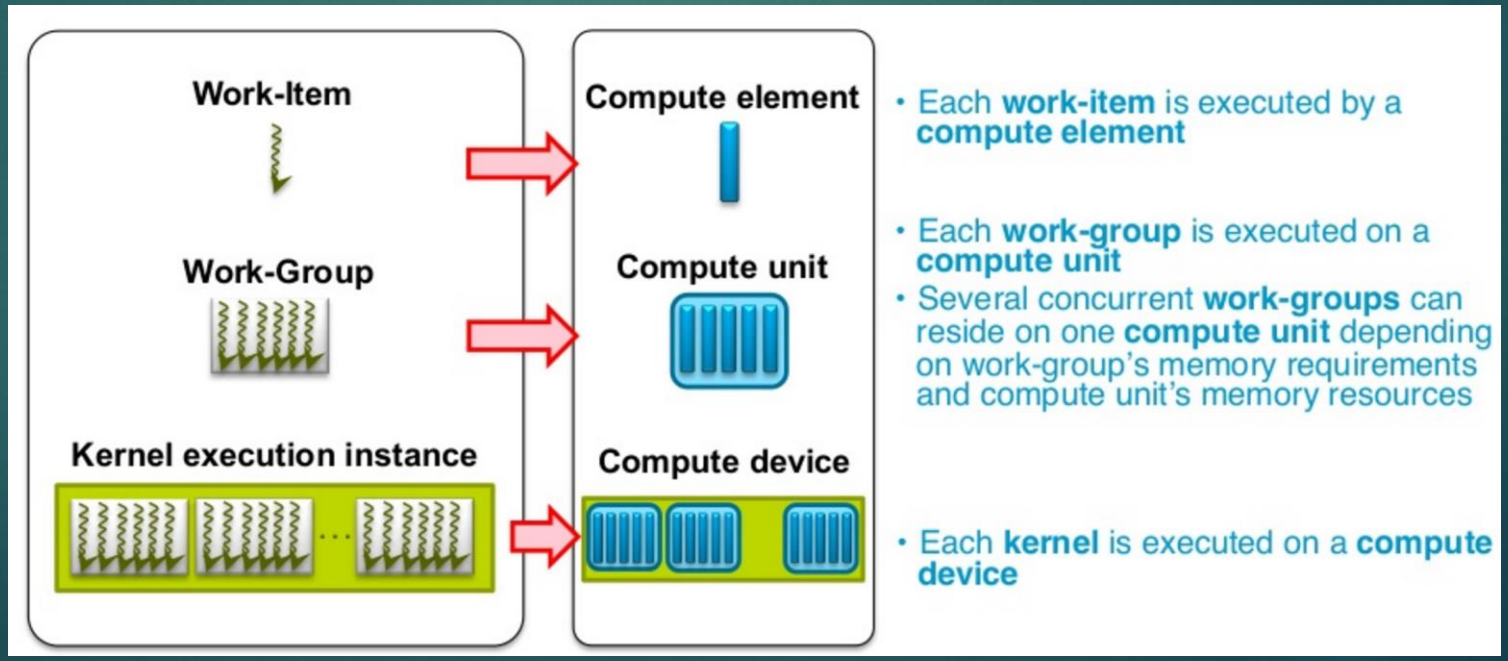
OpenCL Execution Model

- Integrated host+device app C program
  - Serial or modestly parallel parts in **host** C code
  - Highly parallel parts in **device** SPMD kernel C code



Architecture – Execution Model

- Kernel – Smallest unit of execution, like a C function
- Host program – A collection of kernels
- Work item, an instance of kernel at run time
- Work group, a collection of work items

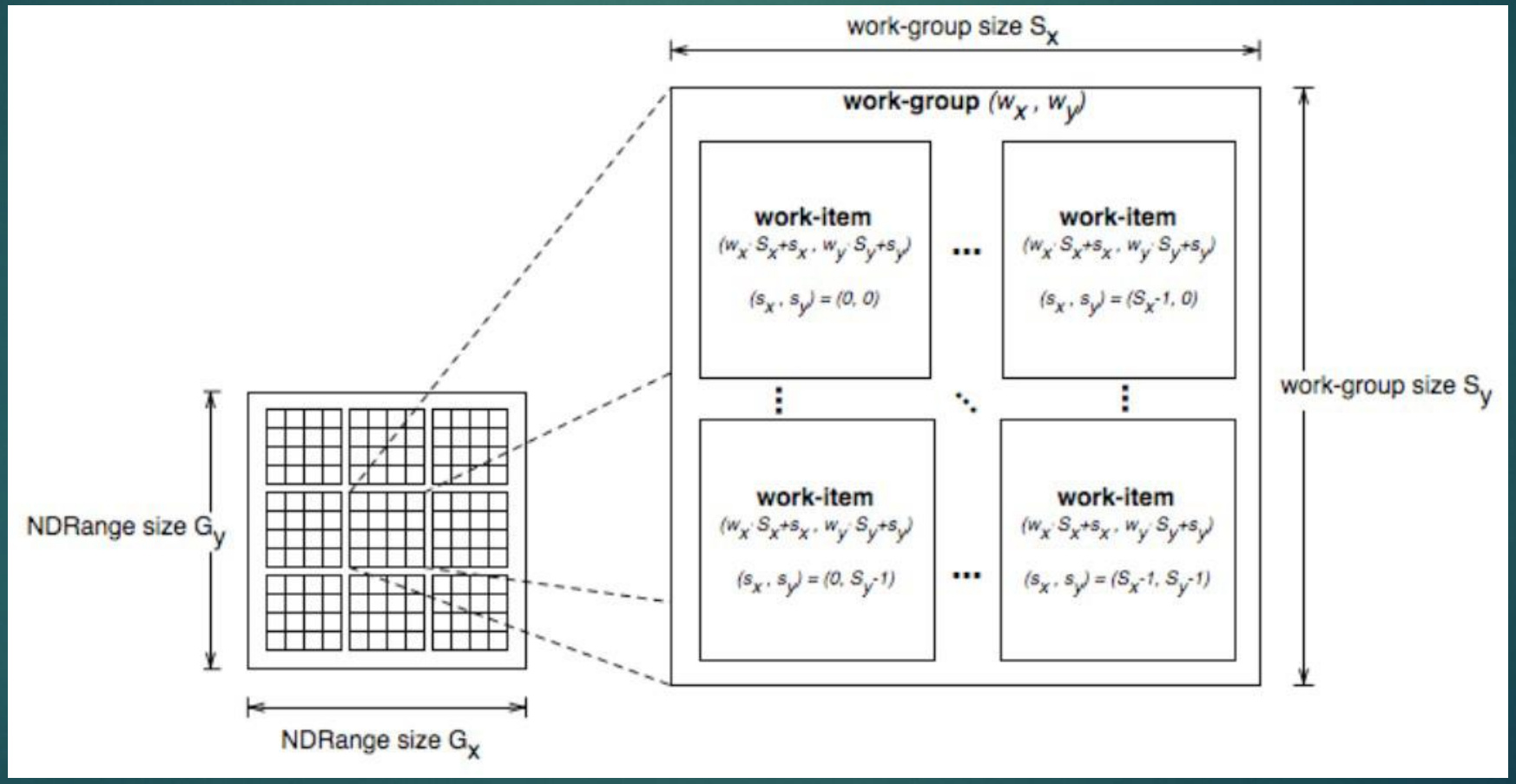


OpenCL Kernels

- Code that actually executes on target devices
- Kernel body is instantiated once for each work item
  - An OpenCL work item is equivalent to a CUDA thread
- Each OpenCL work item gets a unique index

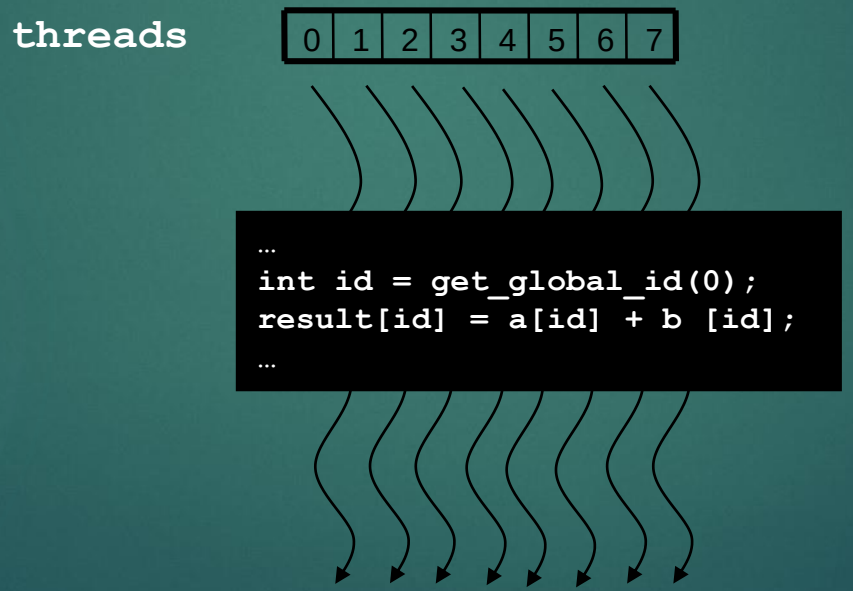
```
__kernel void
vadd(__global const float *a,
     __global const float *b,
     __global float *result) {
  int id = get_global_id(0);
  result[id] = a[id] + b[id];
}
```

Architecture – Execution Model



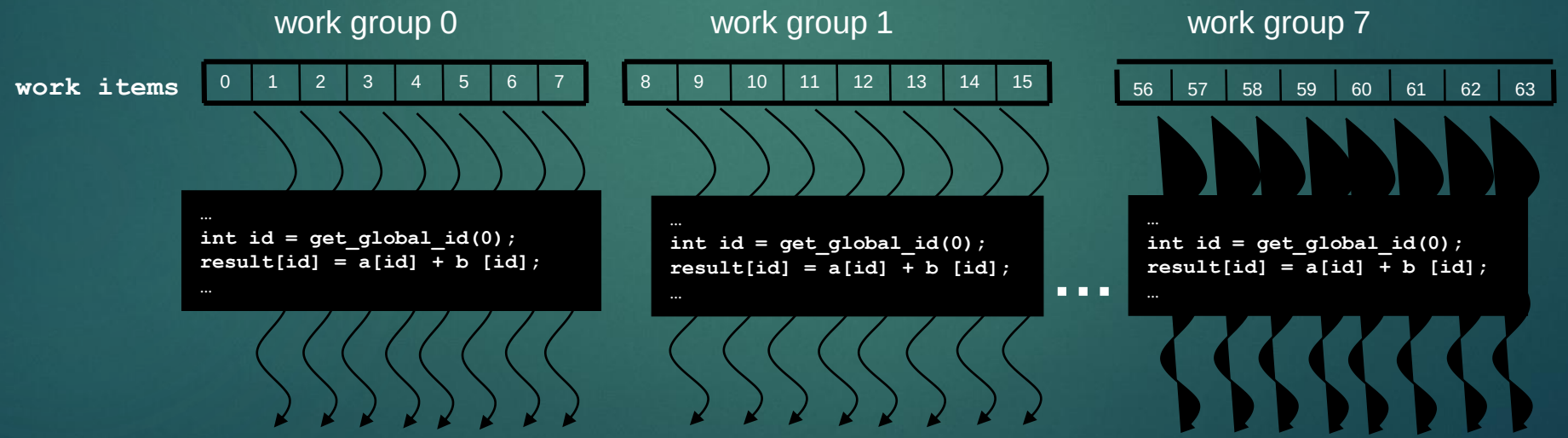
Array of Parallel Work Items

- An OpenCL kernel is executed by an array of work items
  - All work items run the same code (SPMD/SIMD)
  - Each work item has an index that it uses to compute memory addresses and make control decisions



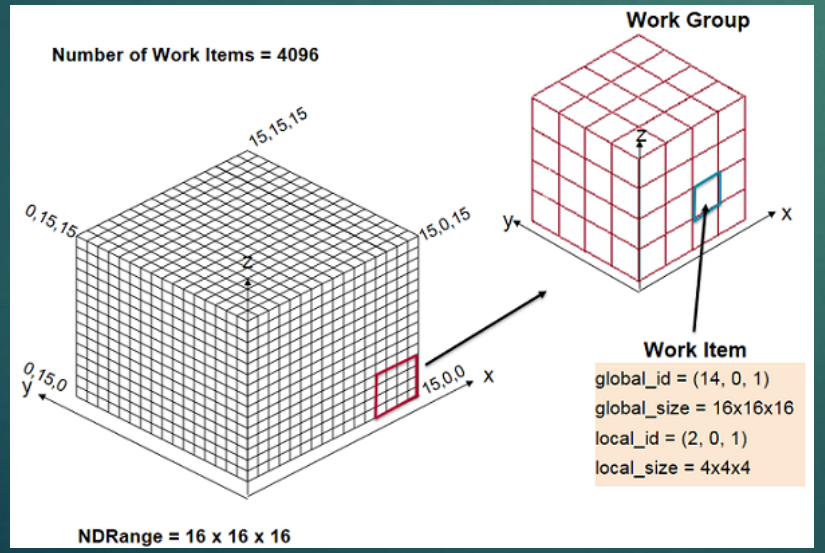
Work Groups: Scalable Cooperation

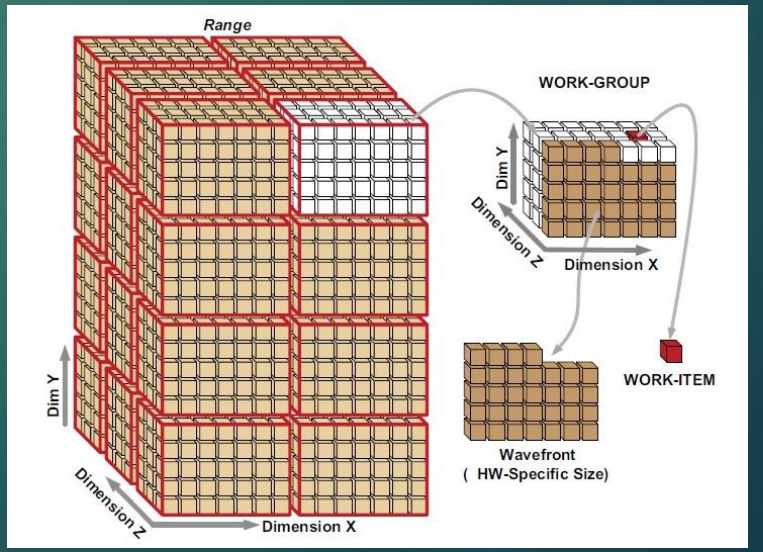
- Divide monolithic work item array into work groups
  - Work items within a work group cooperate via **shared memory**, **atomic operations** and **barrier synchronization**
  - Work items in different work groups cannot cooperate



OpenCL Data Parallel Model Summary

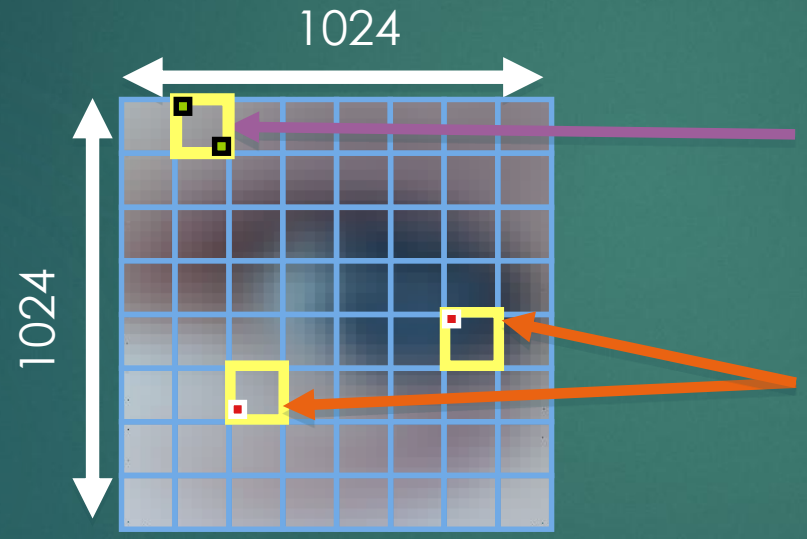
- Parallel work is submitted to devices by launching kernels
- Kernels run over global dimension index ranges (NDRange), broken up into “work groups”, and “work items”
- Work items executing within the same work group can synchronize with each other with barriers or memory fences
- Work items in different work groups can’t sync with each other, except by launching a new kernel





An N-dimensional domain of work-items

- Global Dimensions:
  - 1024x1024 (whole problem space)
- Local Dimensions:
  - 64x64 (**work-group**, executes together)



Synchronization between work-items possible only within work-groups: barriers and memory fences

Cannot synchronize between work-groups within a kernel

- Choose the dimensions that are "best" for your algorithm

OpenCL N Dimensional Range (NDRange)

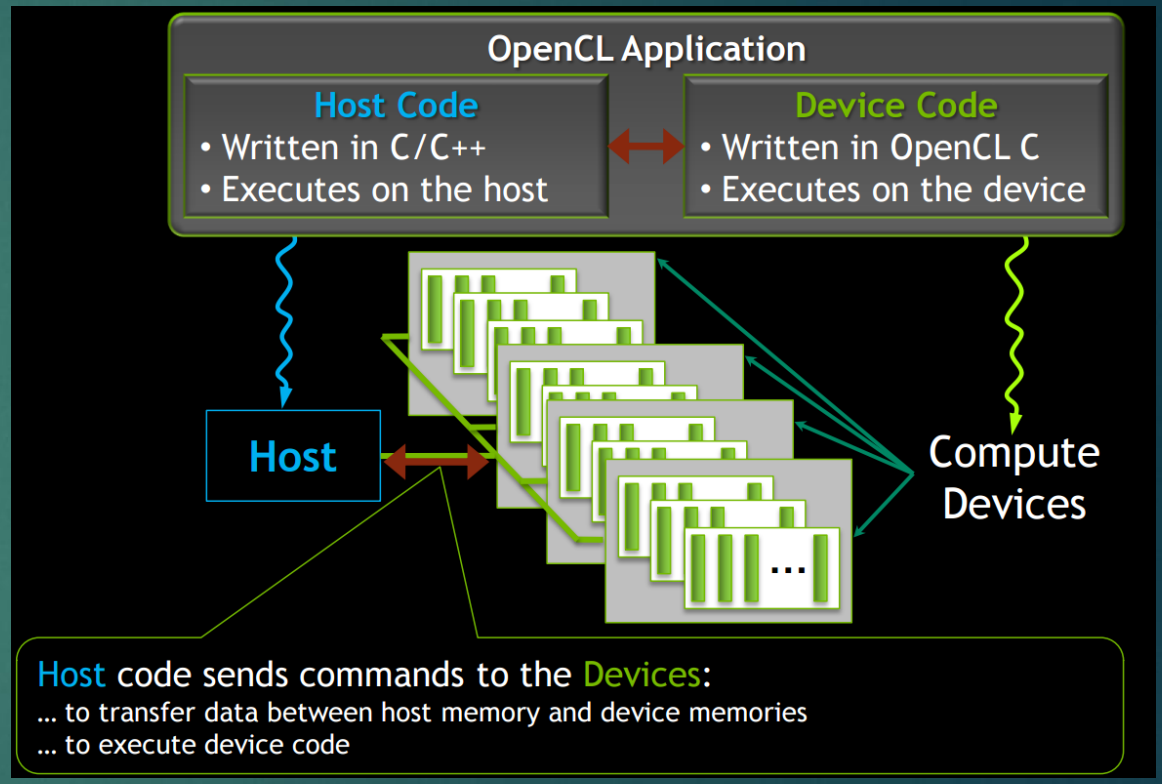
- The problem we want to compute should have some **dimensionality**;
  - For example, compute a kernel on all points in a cube
- When we execute the kernel we specify **up to 3 dimensions**
- We also **specify the total problem size** in each dimension – this is called the **global** size
- We associate each point in the iteration space with a **work-item**

OpenCL N Dimensional Range (NDRange)

- Work-items are grouped into **work-groups**; work-items within a work-group can share **local memory** and can **synchronize**
- We can specify the number of work-items in a work-group – this is called the **local** (work-group) size
- Or the OpenCL run-time can choose the work-group size for you (usually not optimally)

OpenCL Host Code

- Prepare and trigger device code execution
  - Create and manage device context(s) and associate work queue(s), etc…
  - Memory allocations, memory copies, etc
  - Kernel launch
- OpenCL programs are normally compiled entirely at runtime, which must be managed by host code

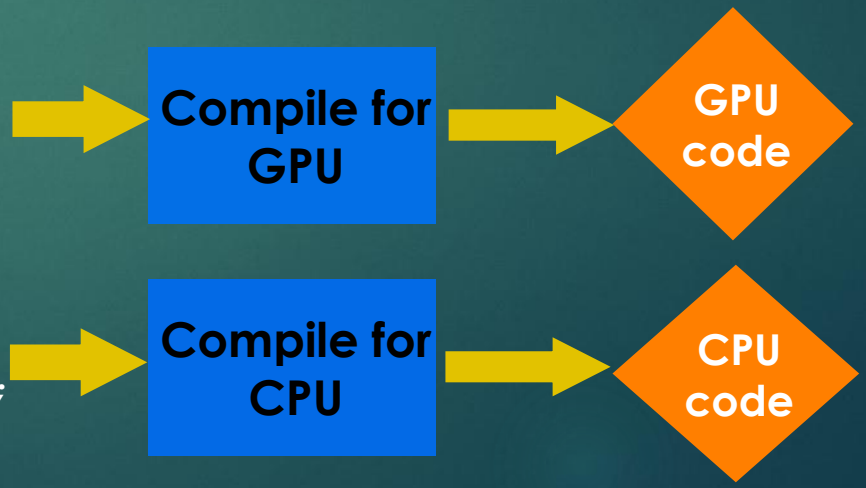


Building Program Objects

- The program object encapsulates:
  - A context
  - The program kernel source or binary
  - List of target devices and build options
- The C API build process to create a program object:
  - **clCreateProgramWithSource()**
  - **clCreateProgramWithBinary()**

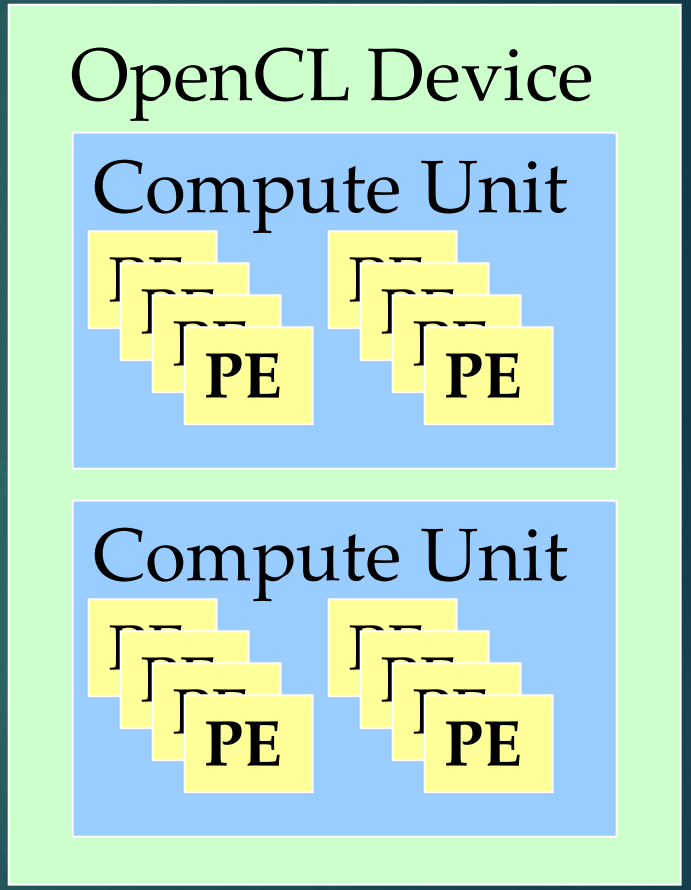
OpenCL uses runtime compilation … because in general you don't know the details of the target device when you ship the program

```
__kernel void
horizontal_reflect(read_only image2d_t src,
                   write_only image2d_t dst)
{
  int x = get_global_id(0);  // x-coord
  int y = get_global_id(1);  // y-coord
  int width = get_image_width(src);
  float4 src_val = read_imagef(src, sampler,
                               (int2)(width-1-x, y));
  write_imagef(dst, (int2)(x, y), src_val);
}
```



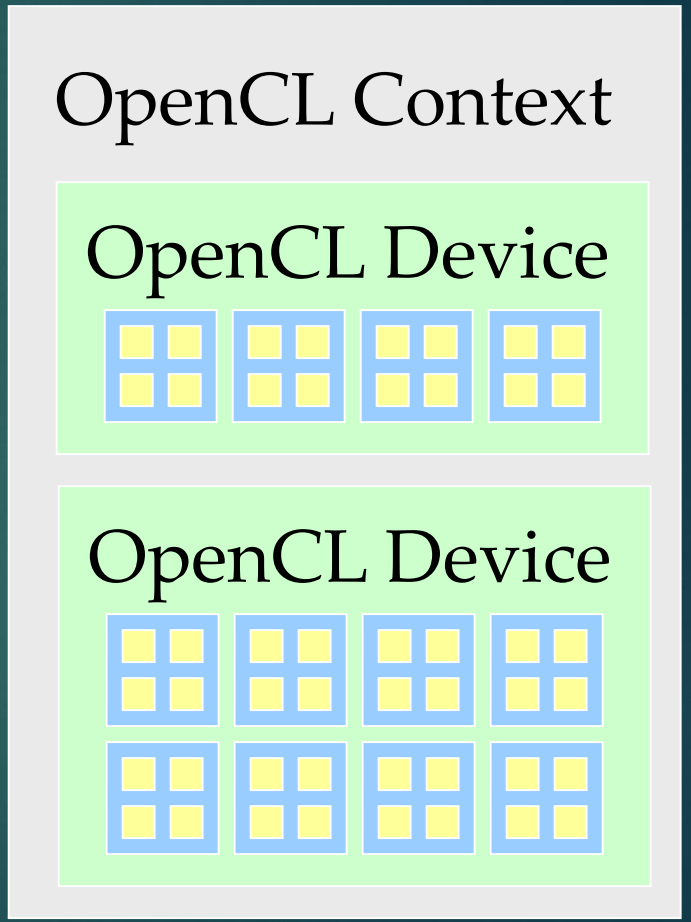
OpenCL Hardware Abstraction

- OpenCL exposes CPUs, GPUs, and other Accelerators as “devices”
- Each “device” contains one or more “compute units”, i.e. cores, SMs, etc…
- Each “compute unit” contains one or more SIMD “processing elements”



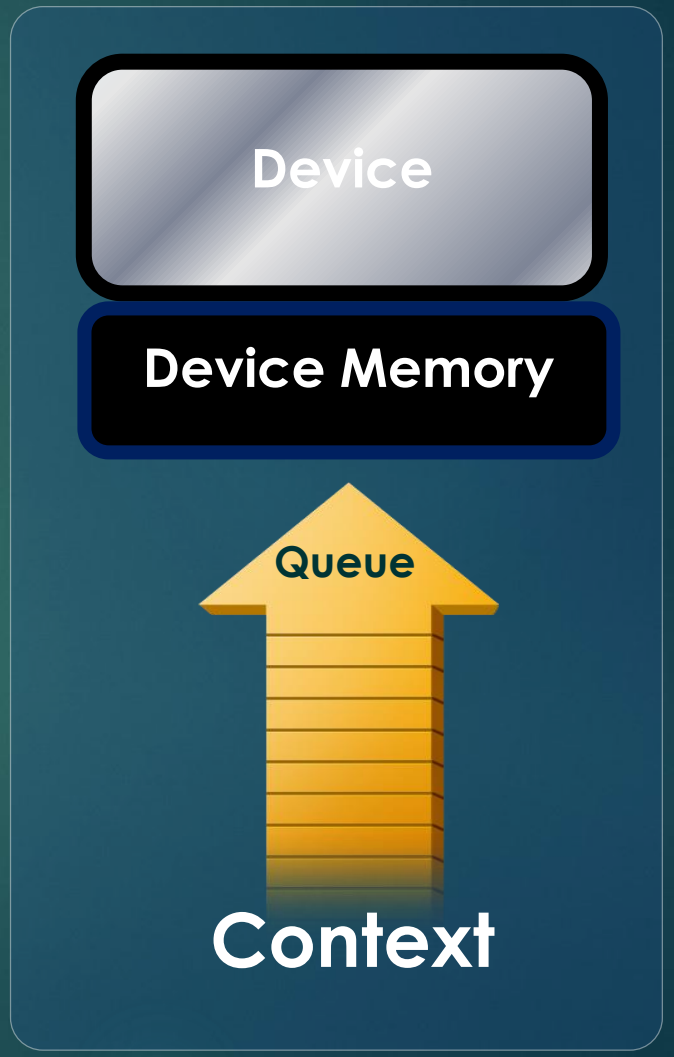
OpenCL Context

- Contains one or more devices
- OpenCL memory objects are associated with a context, not a specific device
- clCreateBuffer() is the main data object allocation function
  - error if an allocation is too large for any device in the context
- Each device needs its own work queue(s)
- Memory transfers are associated with a command queue (thus a specific device)



Context and Command-Queues

- ***Context***:
  - The environment within which kernels execute and in which synchronization and memory management is defined.
- The ***context*** includes:
  - One or more devices
  - Device memory
  - One or more command-queues
- All ***commands*** for a device (kernel execution, synchronization, and memory transfer operations) are submitted through a ***command-queue***.
- Each ***command-queue*** points to a single device within a context.

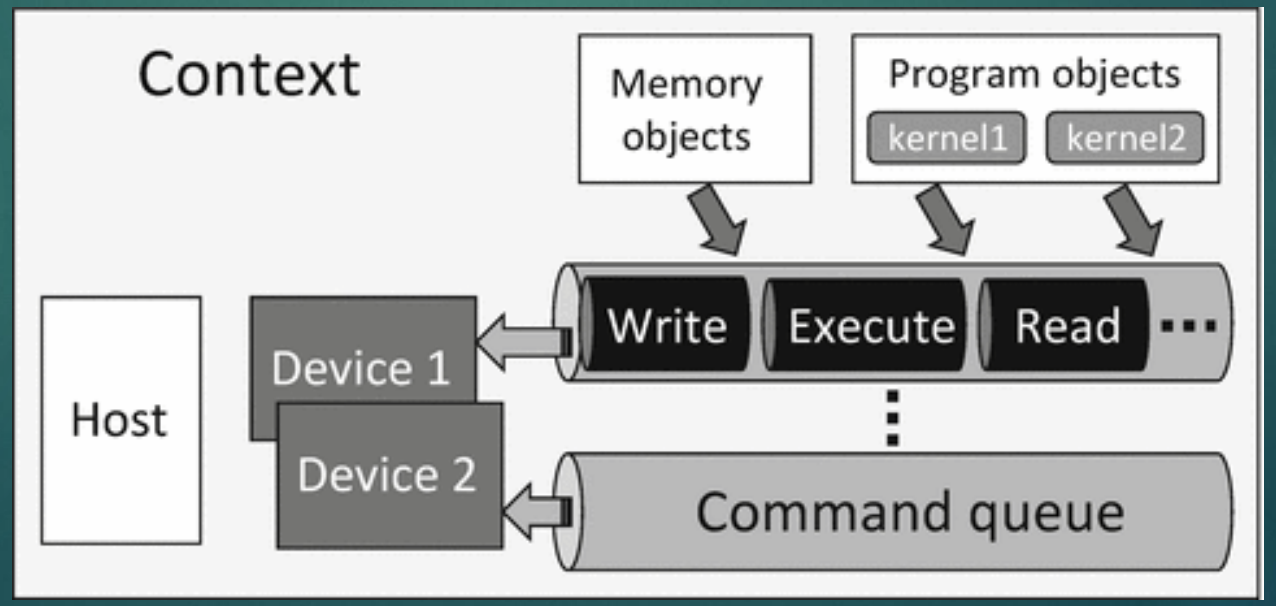


OpenCL Context Setup Code (simple)


```
cl_int clerr = CL_SUCCESS;
cl_context clctx = clCreateContextFromType(0, CL_DEVICE_TYPE_ALL, NULL, NULL, &clerr);

size_t parmsz;
clerr = clGetContextInfo(clctx, CL_CONTEXT_DEVICES, 0, NULL, &parmsz);

cl_device_id* cldevs = (cl_device_id *) malloc(parmsz);
clerr = clGetContextInfo(clctx, CL_CONTEXT_DEVICES, parmsz, cldevs, NULL);

cl_command_queue clcmdq = clCreateCommandQueue(clctx, cldevs[0], 0, &clerr);
```

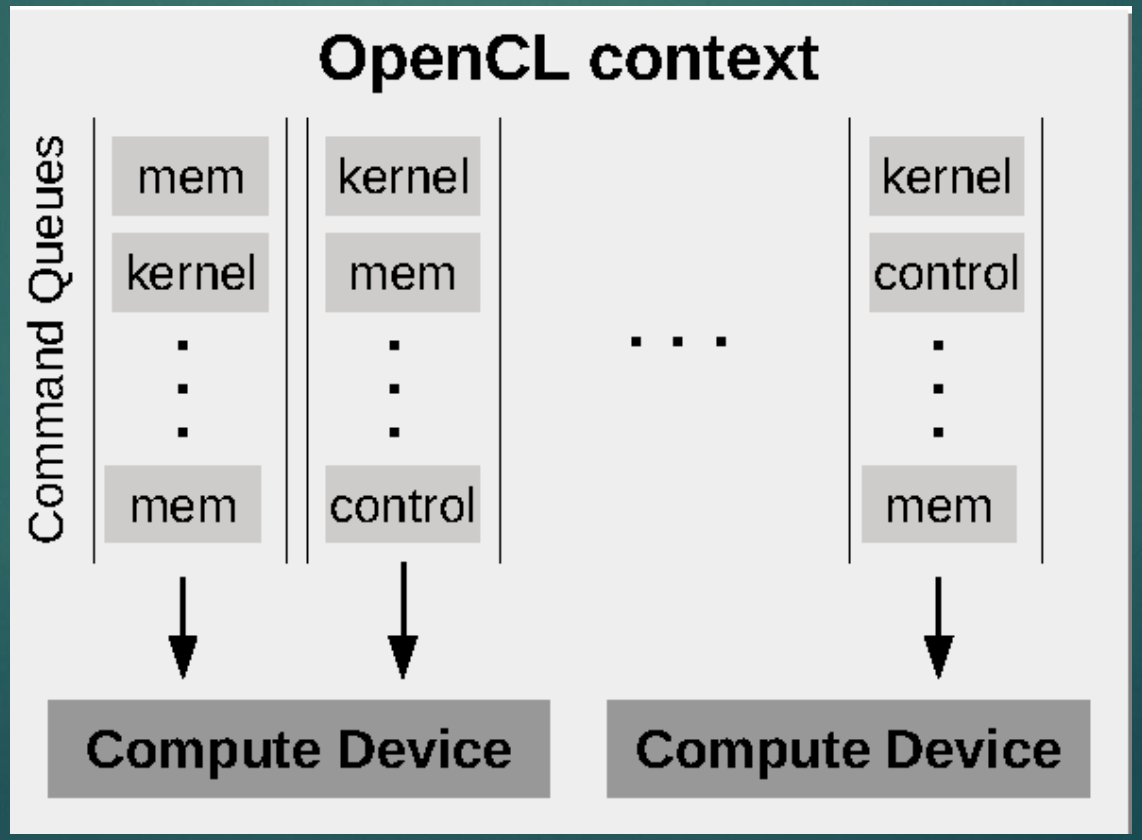
Architecture – Programming Model

- Data Parallel, work group consist of instances of same kernel (work items)
- Different data elements are fed into the work items in the group
- Task Parallel, work group consist of a single work item (instance of kernel)
- Work group can run independently
- Each compute device sees a number of work groups in parallel, thus task parallel

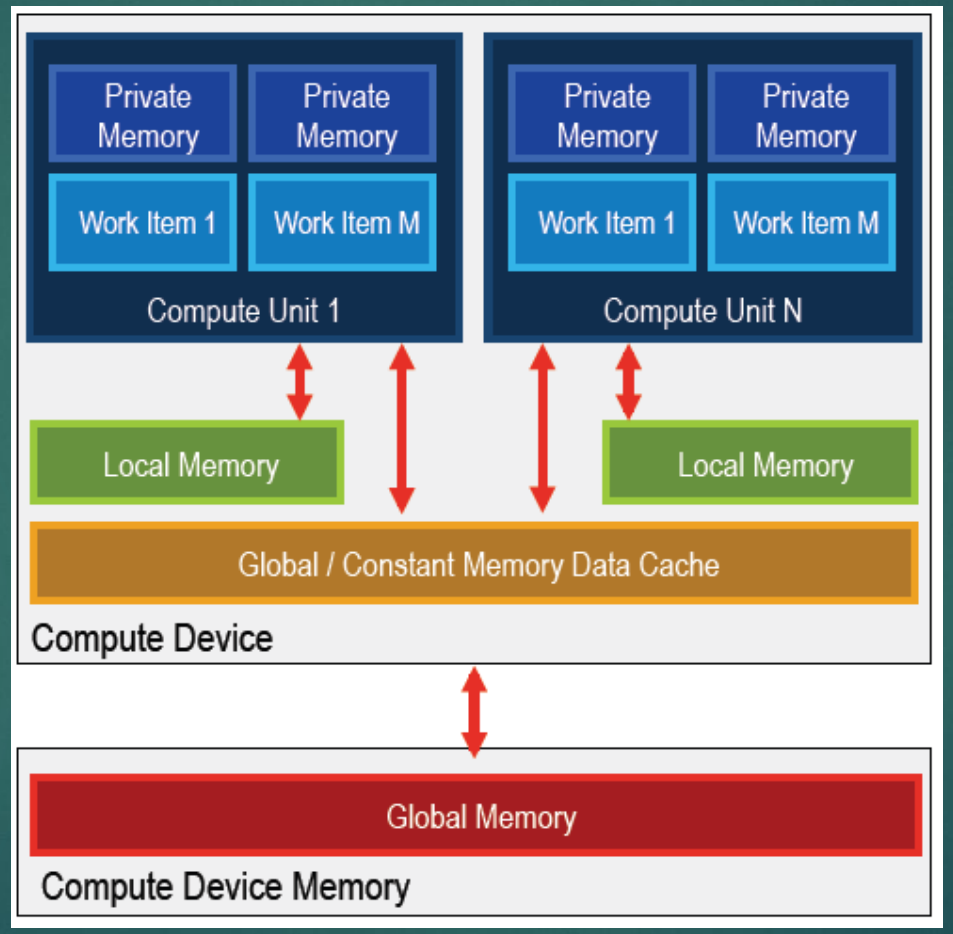


Architecture – Programming Model

- Only CPUs are expected to have task parallel mechanisms
- Data parallel model must be present on all OpenCL compatible devices

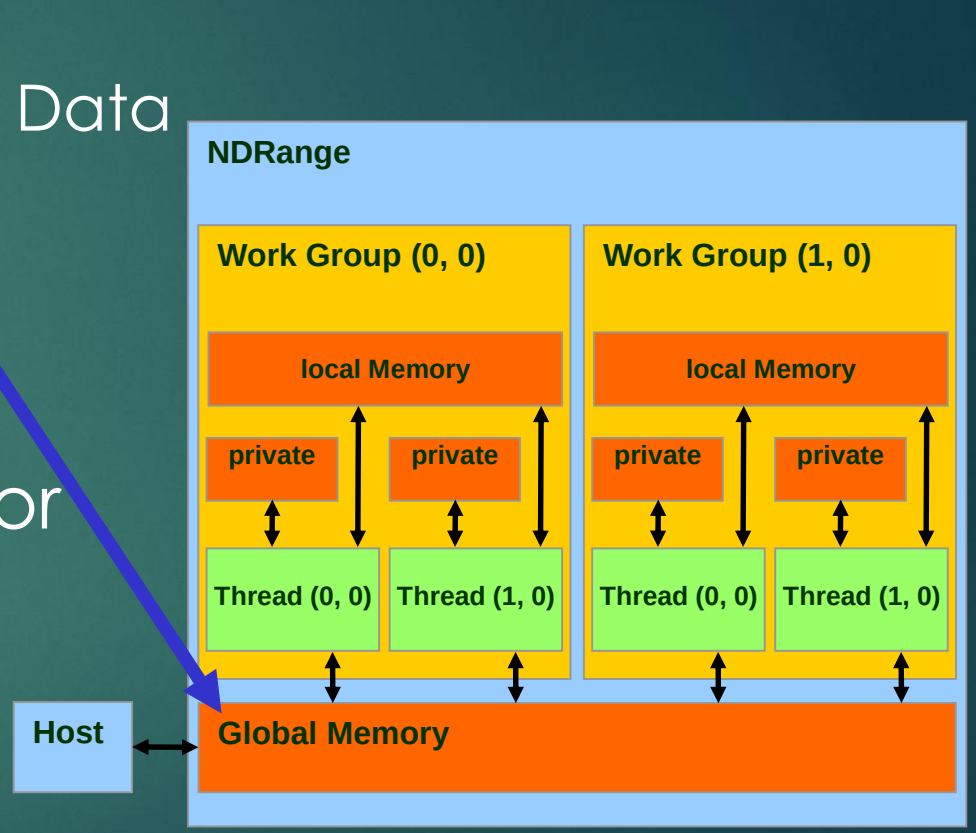


Architecture – Memory Model



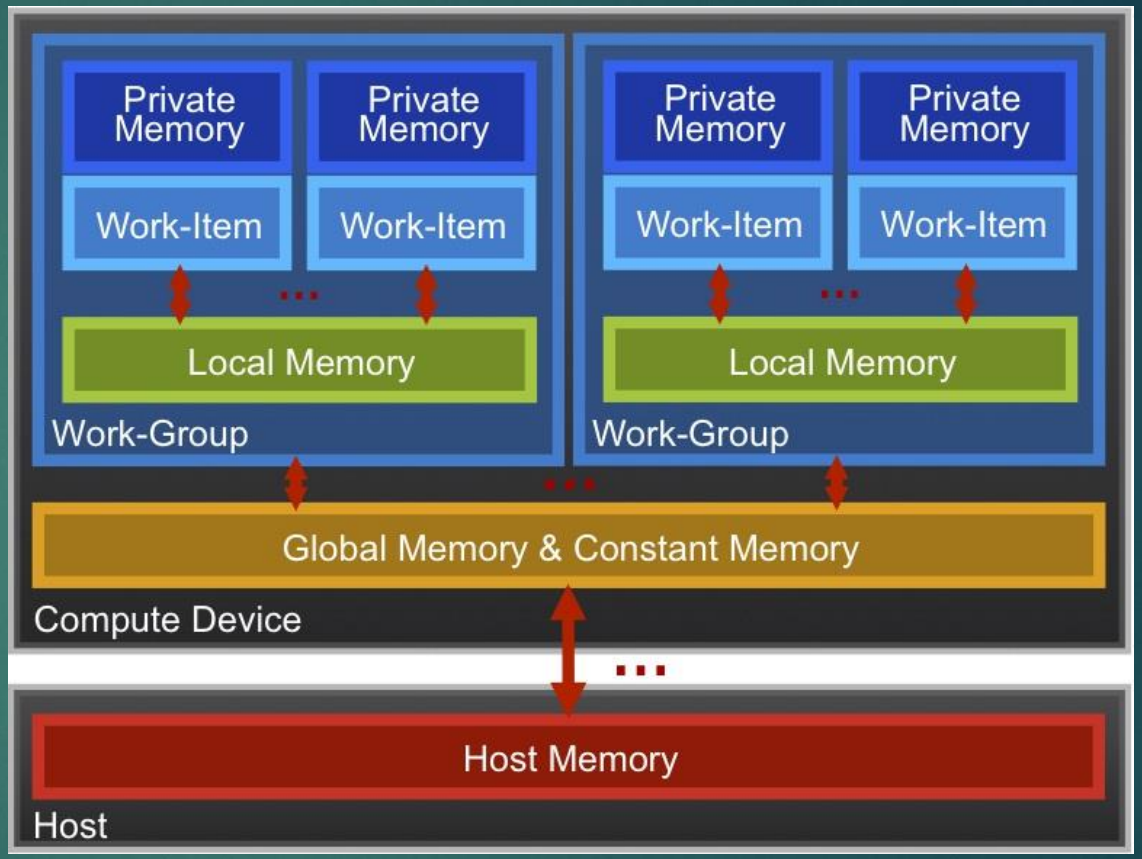
OpenCL Memory Model Overview

- Global memory
  - Main means of communicating R/W Data between host and device
  - Contents visible to all threads
  - Long latency access
- We will focus on global memory for now



OpenCL Memory model

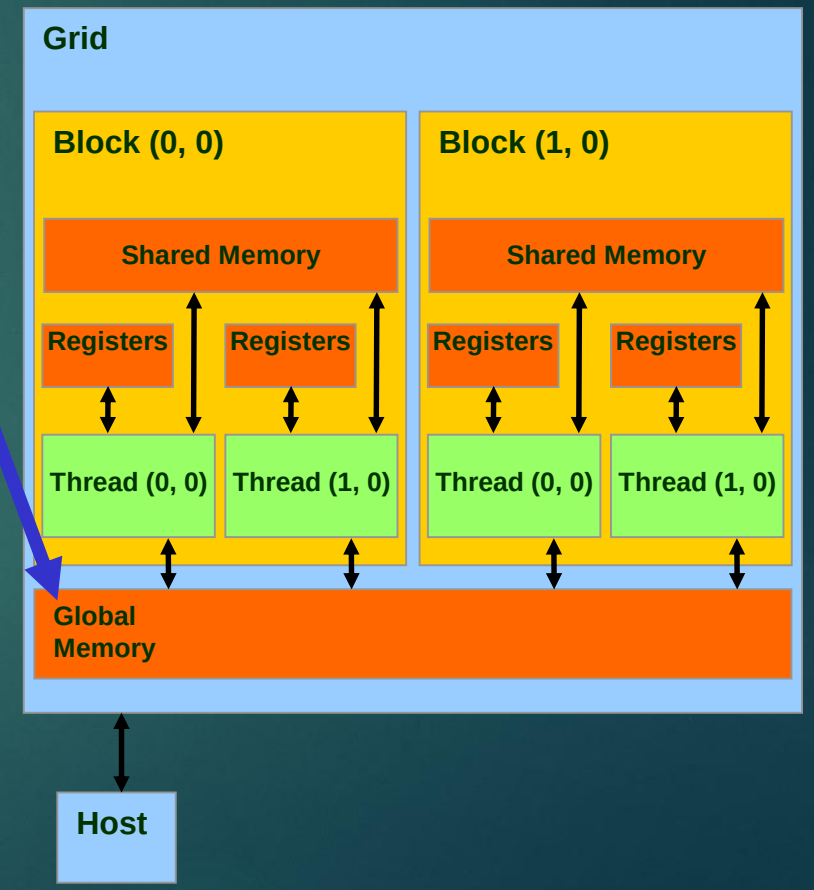
- ***Private Memory***
  - Per work-item
- ***Local Memory***
  - Shared within a work-group
- ***Global Memory /Constant Memory***
  - Visible to all work-groups
- ***Host memory***
  - On the CPU



Memory management is **explicit**:
You are responsible for moving data from
host → global → local *and* back

OpenCL Device Memory Allocation

- **clCreateBuffer();**
  - Allocates object in the device Global Memory
  - Returns a pointer to the object
  - Requires five parameters
    - OpenCL context pointer
    - Flags for access type by device
    - Size of allocated object
    - Host memory pointer, if used in copy-from-host mode
    - Error code
- **clReleaseMemObject()**
  - Frees object
    - Pointer to freed object

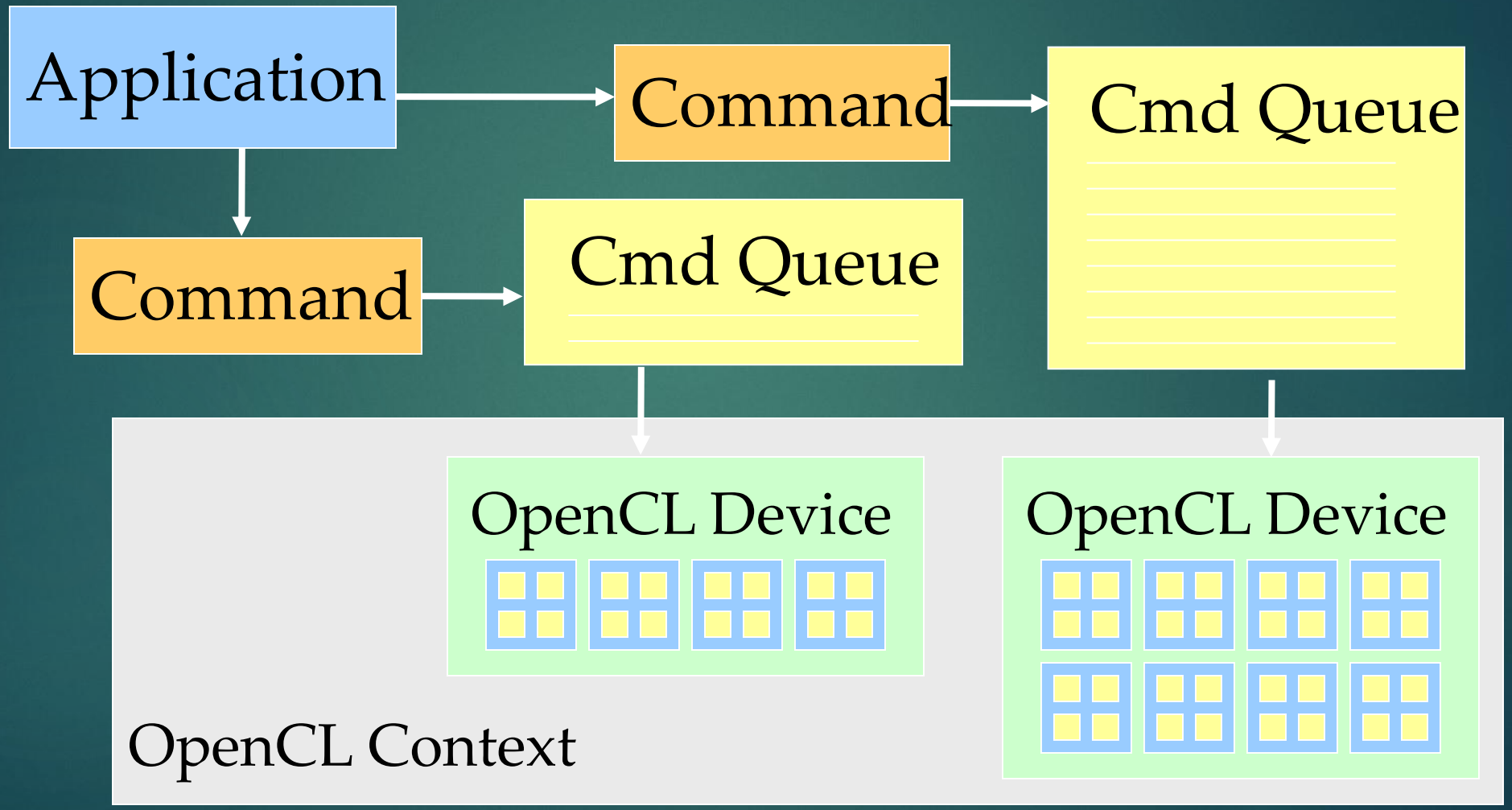


OpenCL Device Memory Allocation (cont.)

- Code example:
  - Allocate a 1024 single precision float array
  - Attach the allocated storage to d_a
  - “d” is often used to indicate a device data structure

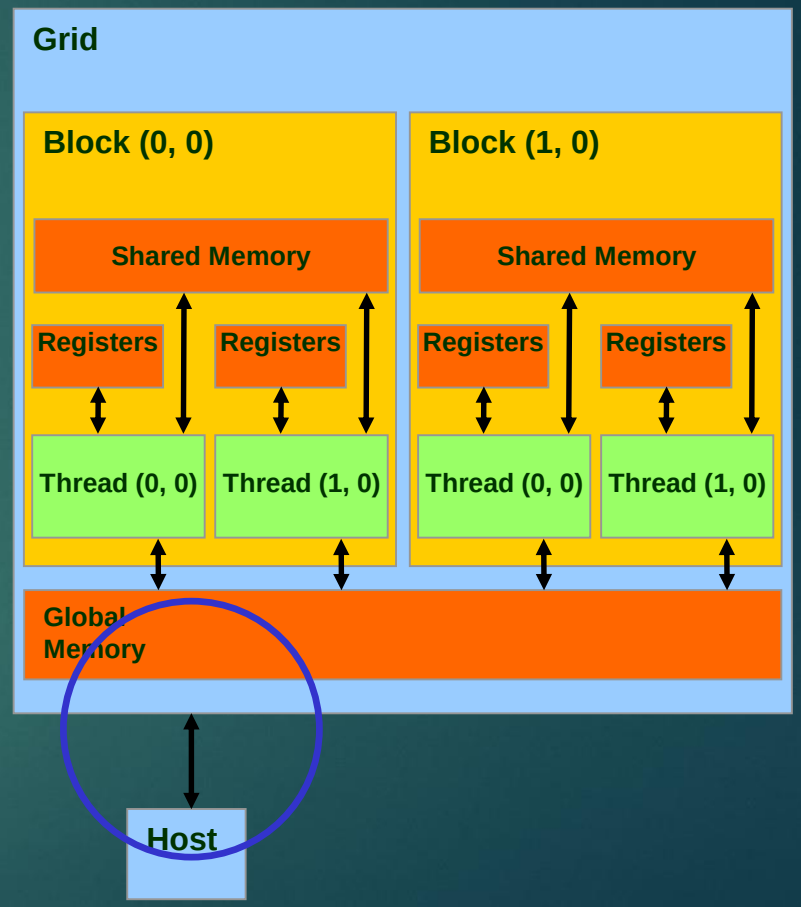
```
VECTOR_SIZE = 1024;
cl_mem d_a;
int size = VECTOR_SIZE* sizeof(float);

d_a = clCreateBuffer(clctx, CL_MEM_READ_ONLY, size, NULL, NULL);
clReleaseMemObject(d_a);
```

OpenCL Device Command Execution



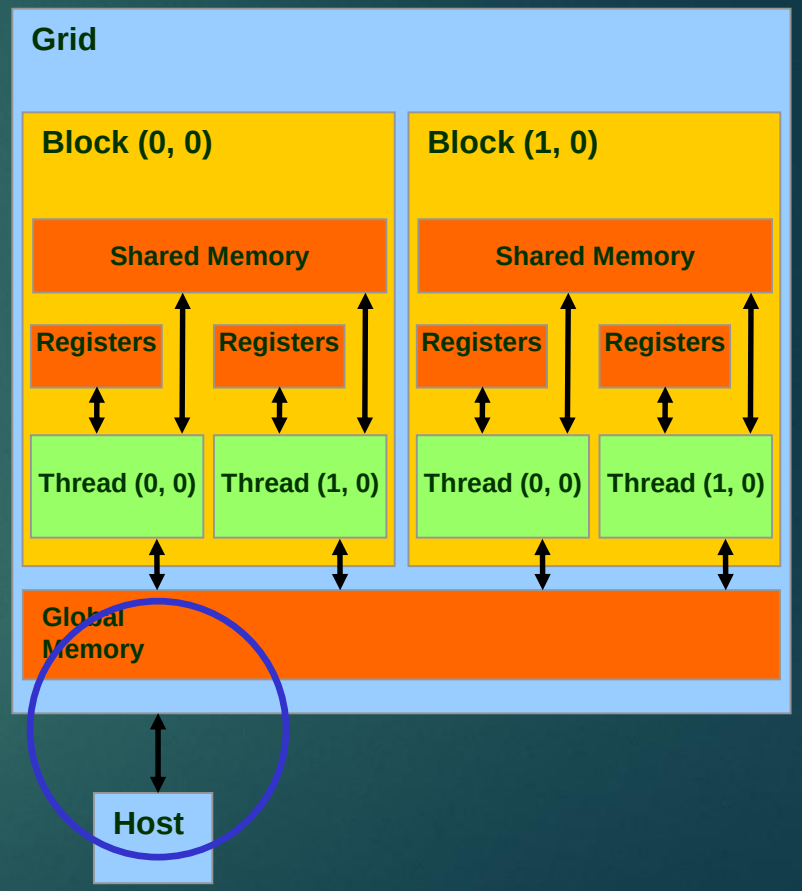
OpenCL Host-to-Device Data Transfer

- clEnqueueWriteBuffer();
  - memory data transfer to device
  - Requires nine parameters
    - OpenCL command queue pointer
    - Destination OpenCL memory buffer
    - Blocking flag
    - Offset in bytes
    - Size of bytes of written data
    - Host memory pointer
    - List of events to be completed before execution of this command
    - Event object tied to this command
- Asynchronous transfer later



OpenCL Device-to-Host Data Transfer

- clEnqueueReadBuffer();
  - memory data transfer to host
  - Requires nine parameters
    - OpenCL command queue pointer
    - Source OpenCL memory buffer
    - Blocking flag
    - Offset in bytes
    - Size of bytes of read data
    - Destination host memory pointer
    - List of events to be completed before execution of this command
    - Event object tied to this command
- Asynchronous transfer later



OpenCL Host-Device Data Transfer (cont.)

- Code example:
  - Transfer a 64 * 64 single precision float array
  - a is in host memory and d_a is in device memory

```
clEnqueueWriteBuffer(clcmdq, d_a, CL_FALSE, 0,
      mem_size, (const void * )a, 0, 0, NULL);

clEnqueueReadBuffer(clcmdq, d_result, CL_FALSE, 0,
      mem_size, (void * ) host_result, 0, 0, NULL);
```

OpenCL Host-Device Data Transfer (cont.)

- clCreateBuffer and clEnqueueWriteBuffer can be combined into a single command using special flags.
- Eg:

```
d_A=clCreateBuffer(clctxt, CL_MEM_READ_ONLY |
  CL_MEM_COPY_HOST_PTR, mem_size, h_A, NULL);
```

  - Combination of 2 flags here. CL_MEM_COPY_HOST_PTR to be used only if a valid host pointer is specified.
  - This creates a memory buffer on the device, and copies data from h_A into d_A.
  - Includes an implicit clEnqueueWriteBuffer operation, for all devices/command queues tied to the context clctxt.

OpenCL Memory Systems

- __global – large, long latency
- __private – on-chip device registers
- __local – memory accessible from multiple PEs or work items. May be SRAM or DRAM, must query…
- __constant – read-only constant cache
- Device memory is managed explicitly by the programmer, as with CUDA

OpenCL Kernel Compilation Example

OpenCL kernel source code as a big string

```
const char* vaddsrc =
  "__kernel void vadd(__global float *d_A, __global float *d_B, __global float *d_C, int N) { \n"   […etc and so forth…]
cl_program clpgm;
clpgm = clCreateProgramWithSource(clctx, 1, &vaddsrc, NULL, &clerr);
char clcompileflags[4096];
sprintf(clcompileflags, "-cl-mad-enable");
clerr = clBuildProgram(clpgm, 0, NULL, clcompileflags, NULL, NULL);
cl_kernel clkern = clCreateKernel(clpgm, "vadd", &clerr);
```

Gives raw source code string(s) to OpenCL

Set compiler flags, compile source, and retreive a handle to the "vadd" kernel

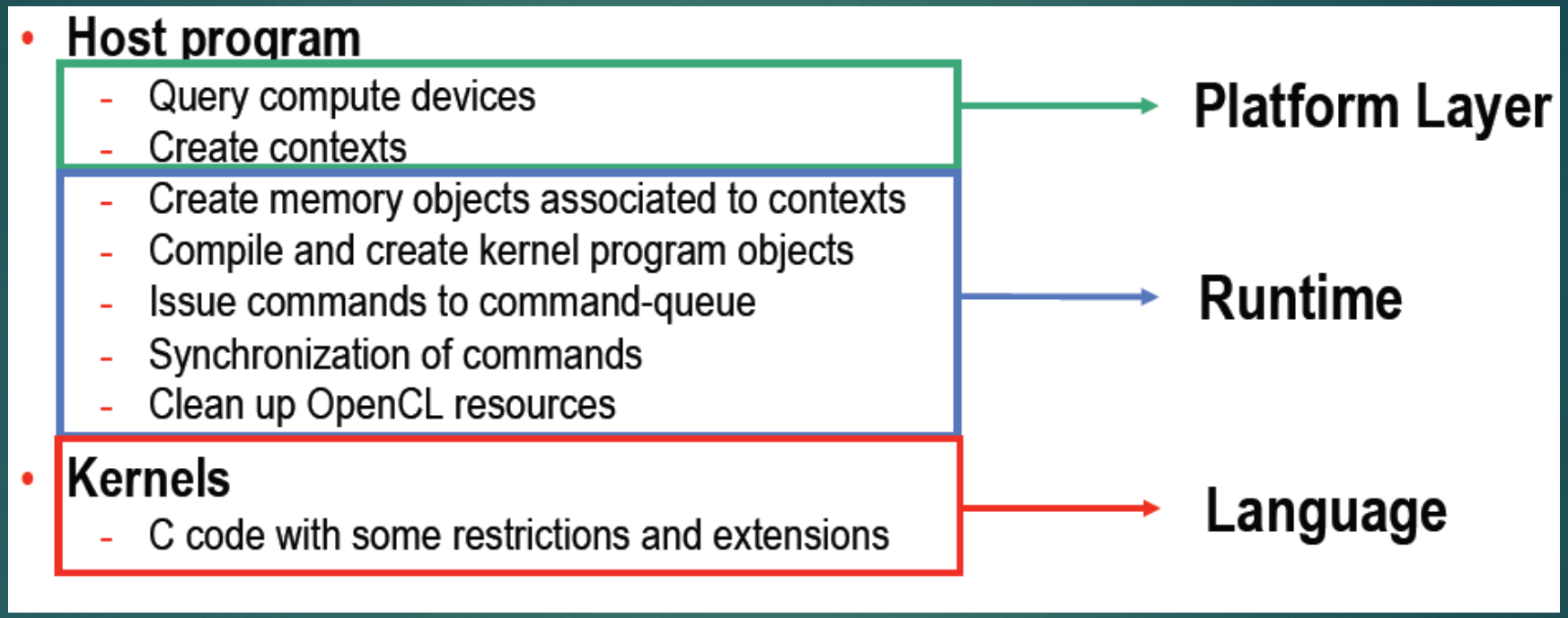
Summary: Host code for vadd


```
int main()
{   // allocate and initialize host (CPU) memory
    float *h_A = …,   *h_B = …;
    // allocate device (GPU) memory
   cl_mem d_A, d_B, d_C;
   d_A = clCreateBuffer(clctx, CL_MEM_READ_ONLY |
          CL_MEM_COPY_HOST_PTR, N *sizeof(float), h_A, NULL);
   d_B = clCreateBuffer(clctx, CL_MEM_READ_ONLY |
          CL_MEM_COPY_HOST_PTR, N *sizeof(float), h_B, NULL);
   d_C = clCreateBuffer(clctx, CL_MEM_WRITE_ONLY, N *sizeof(float), NULL, NULL);
   clkern=clCreateKernel(clpgm, "vadd", NULL);
   clerr= clSetKernelArg(clkern, 0,sizeof(cl_mem), (void *) &d_A);
   clerr= clSetKernelArg(clkern, 1,sizeof(cl_mem), (void *) &d_B);
   clerr= clSetKernelArg(clkern, 2,sizeof(cl_mem), (void *) &d_C);
   clerr= clSetKernelArg(clkern, 3,sizeof(int), &N);
   cl_event event=NULL;
     clerr= clEnqueueNDRangeKernel(clcmdq,clkern, 2, NULL, Gsz,Bsz, 0, NULL, &event);
     clerr= clWaitForEvents(1, &event);
   clEnqueueReadBuffer(clcmdq, d_C, CL_TRUE, 0, N*sizeof(float), h_C, 0, NULL, NULL);
     clReleaseMemObject(d_A);
     clReleaseMemObject(d_B);
     clReleaseMemObject(d_C);
}
```

OpenCL Software: Runtime

- Language derived from ISO C99 (C Language)
- Restrictions:
  - No recursion
  - no function points
- All standard data types, including vectors
- OpenGL extension

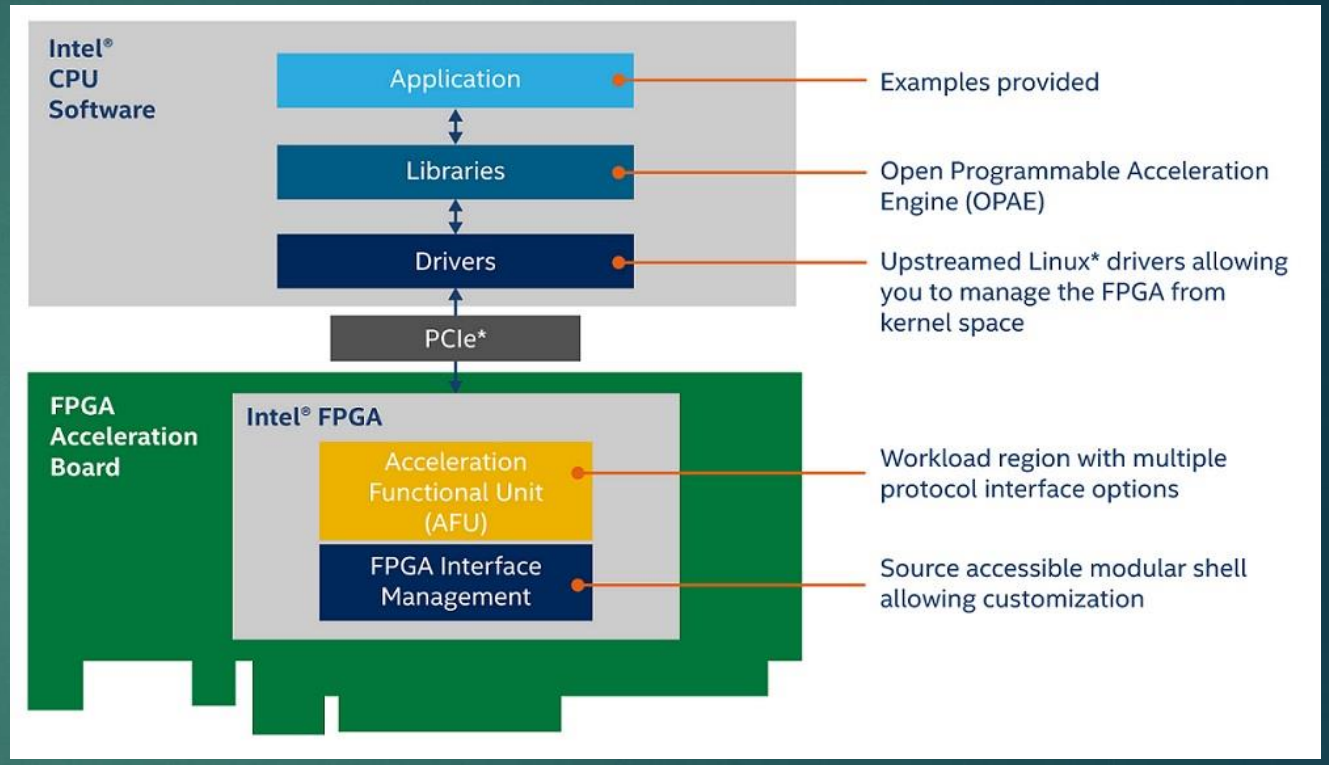
OpenCL Software Stack



- Shows the steps to develop an OpenCL program

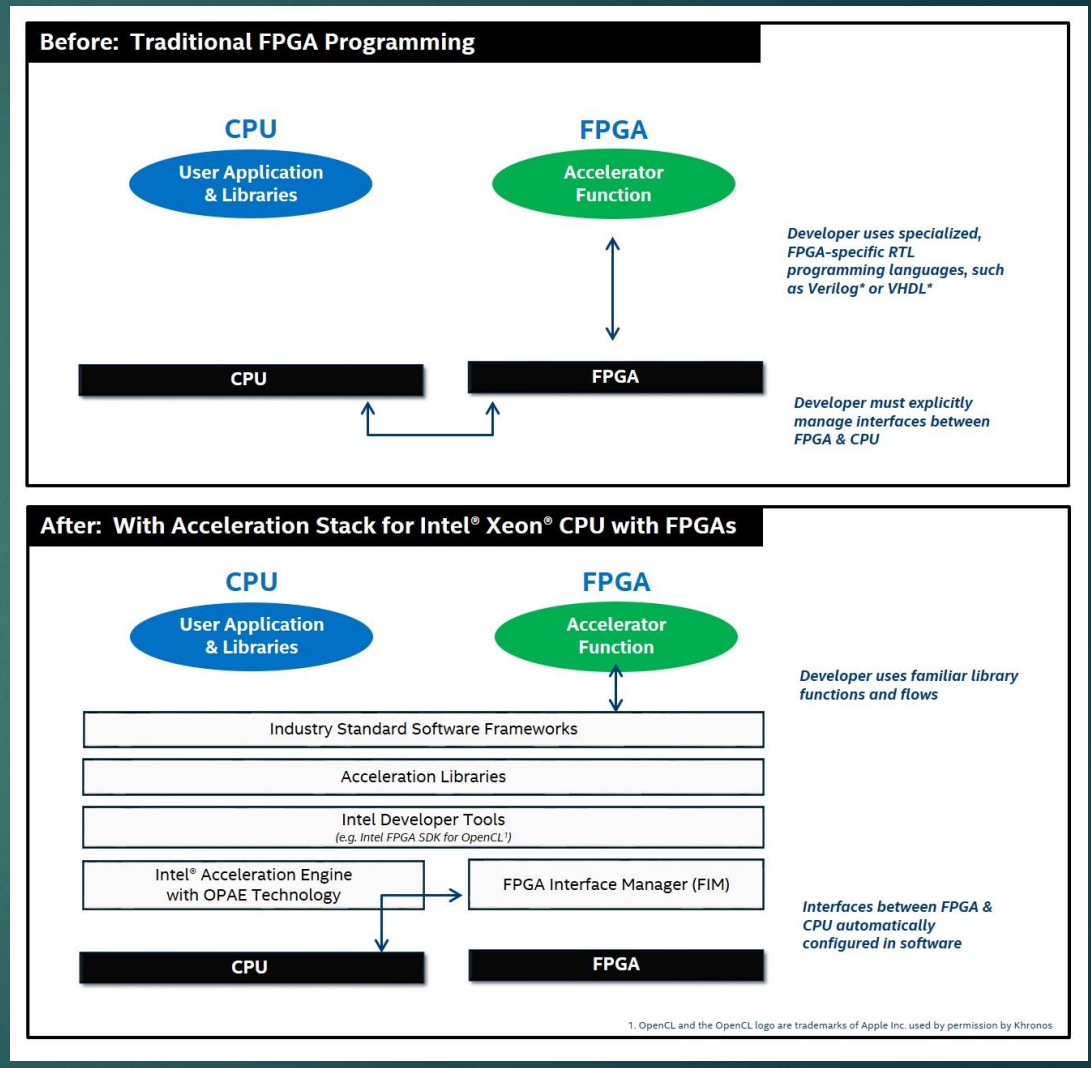
OpenCL Software Environments

- Drivers
  - Nvidia
  - AMD
  - Intel
  - IBM
- Libraries
  - OpenCL toolbox for MATLAB
  - OpenCLLink for Mathematica
  - OpenCL Data Parallel Primitives Library (clpp)
  - ViennaCL – linear algebra library



OpenCL Software Environments

- Other language bindings
  - WebCL JavaScript Firefox and WebKit
  - Python PyOpenCL
  - The Open Toolkit library – C#, OpenGL, OpenAL, Mono/.NET
  - Fortran
- Tools
  - gDEBugger
  - clcc
  - SHOC (Scalable Heterogeneous Computing Benchmark Suite)
  - ImageMagick



OpenCL Example: vector addition

- The "hello world" program of data parallel programming is a program to add two vectors

```
C[i] = A[i] + B[i] for i=0 to N-1
```

- For the OpenCL solution, there are two parts
  - Kernel code
  - Host code

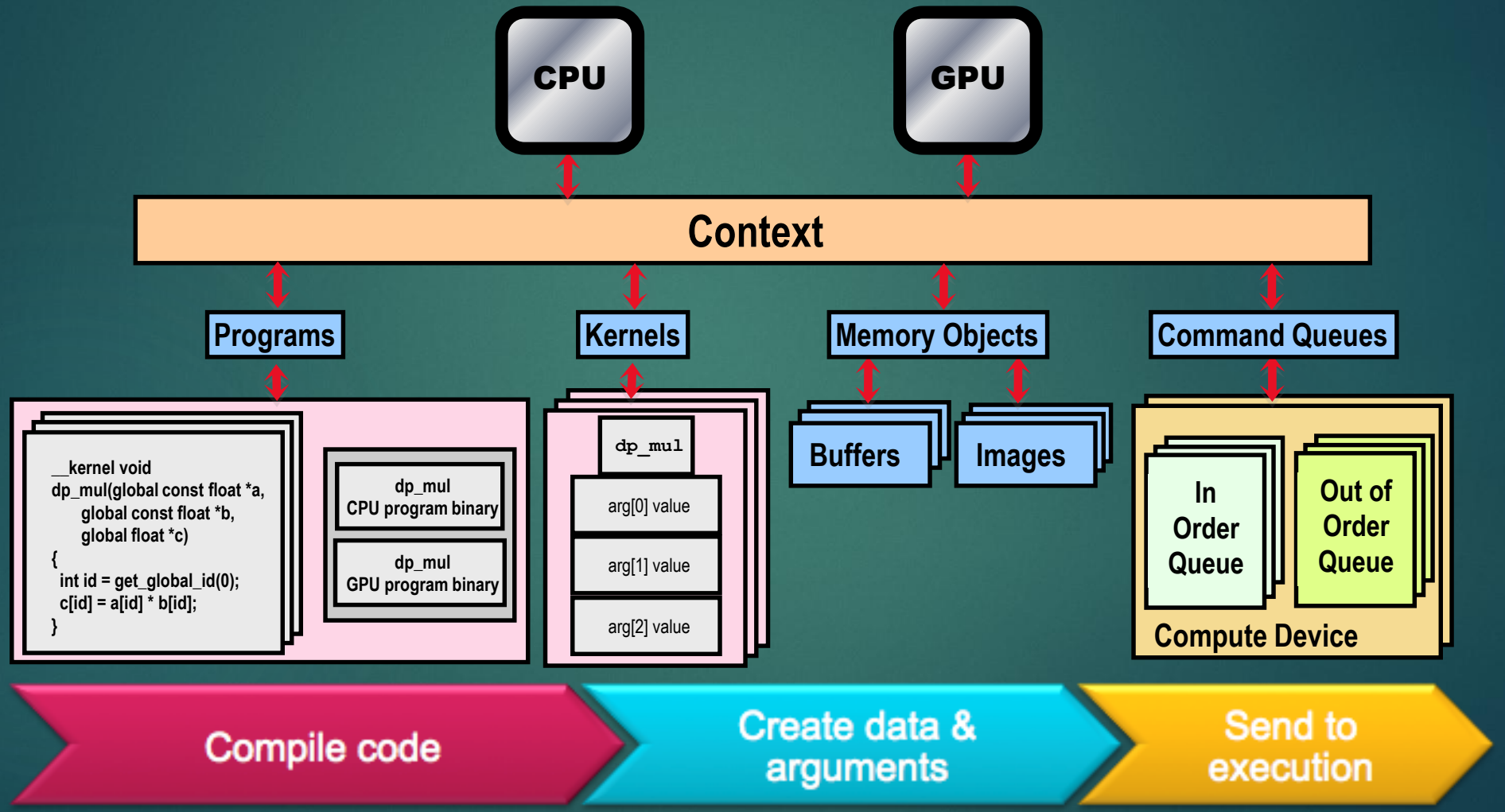
Vector Addition - Kernel


```
__kernel void vadd(__global const float *a,
                   __global const float *b,
                   __global       float *c)
{
    int gid = get_global_id(0);
    c[gid]  = a[gid] + b[gid];
}
```

Vector Addition – Host

- The host program is the code that runs on the host to:
  - Setup the environment for the OpenCL program
  - Create and manage kernels
- 5 simple steps in a basic host program:
  1. Define the ***platform*** … platform = devices+context+queues
  2. Create and Build the ***program*** (dynamic library for kernels)
  3. Setup ***memory*** objects
  4. Define the ***kernel*** (attach arguments to kernel functions)
  5. Submit ***commands*** … transfer memory objects and execute kernels

The basic platform and runtime APIs in OpenCL (using C)



1. Define the platform

- Grab the first available platform:

```
err = clGetPlatformIDs(1, &firstPlatformId, &numPlatforms);
```

- Use the first CPU device the platform provides:

```
err = clGetDeviceIDs(firstPlatformId, CL_DEVICE_TYPE_CPU, 1,
&device_id, NULL);
```

- Create a simple context with a single device:

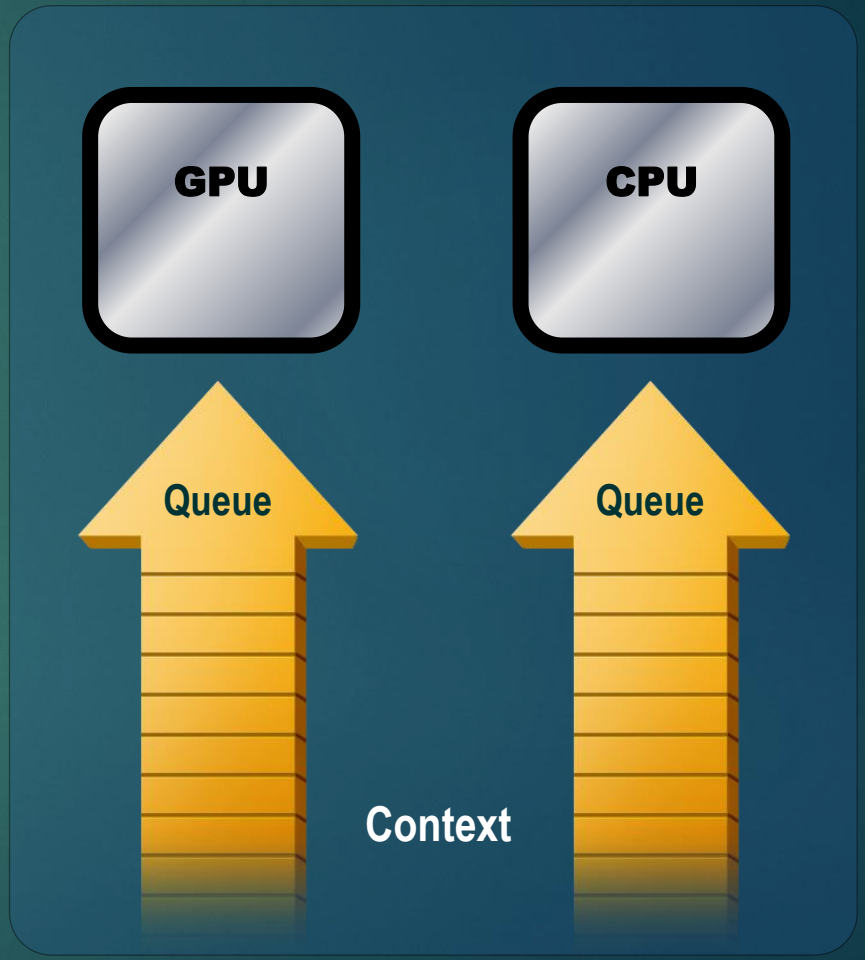
```
context = clCreateContext(firstPlatformId, 1, &device_id, NULL, NULL,
&err);
```

- Create a simple command-queue to feed our device:

```
commands = clCreateCommandQueue(context, device_id, 0, &err);
```

Command-Queues

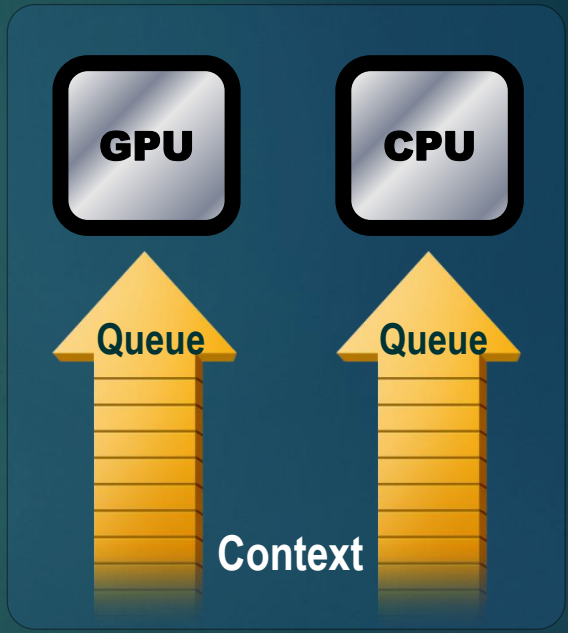
- Commands include:
  - Kernel executions
  - Memory object management
  - Synchronization
- The only way to submit commands to a device is through a command-queue.
- Each command-queue points to a single device within a context.
- Multiple command-queues can feed a single device.
  - Used to define independent streams of commands that don't require synchronization



Command-Queue execution details

Command queues can be configured in different ways to control how commands execute

- ***In-order queues***:
  - Commands are enqueued and complete in the order they appear in the program (program-order)
- ***Out-of-order queues***:
  - Commands are enqueued in program-order but can execute (and hence complete) in any order.
- Execution of commands in the command-queue are guaranteed to be completed at synchronization points



2. Create and Build the program

- Define source code for the kernel-program as a string literal (great for toy programs) or read from a file (for real applications).

- Build the program object:

```
program = clCreateProgramWithSource(context, 1
            (const char**) &KernelSource, NULL, &err);
```

- Compile the program to create a "dynamic library" from which specific kernels can be pulled:

```
err = clBuildProgram(program, 0, NULL,NULL,NULL,NULL);
```

Error messages

- Fetch and print error messages:

```
if (err != CL_SUCCESS) {
 size_t len;
 char buffer[2048];
 clGetProgramBuildInfo(program, device_id,
  CL_PROGRAM_BUILD_LOG, sizeof(buffer), buffer, &len);
 printf("%s\n", buffer);
}
```

- Important to do check all your OpenCL API error messages!
- Easier in C++ with try/catch (see later)

3. Setup Memory Objects

- For vector addition we need 3 memory objects, one each for input vectors A and B, and one for the output vector C.
- Create input vectors and assign values on the host:

```
float h_a[LENGTH], h_b[LENGTH], h_c[LENGTH];
for (i = 0; i < length; i++) {
    h_a[i] = rand() / (float)RAND_MAX;
    h_b[i] = rand() / (float)RAND_MAX;
}
```

- Define OpenCL memory objects:

```
d_a = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(float)*count, NULL, NULL);
d_b = clCreateBuffer(context, CL_MEM_READ_ONLY, sizeof(float)*count, NULL, NULL);
d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, sizeof(float)*count, NULL, NULL);
```

What do we put in device memory?

Memory Objects:

- A handle to a reference-counted region of global memory.

There are two kinds of memory object

- ***Buffer*** object:
  - Defines a linear collection of bytes ("*just a C array*").
  - The contents of buffer objects are fully exposed within kernels and can be accessed using pointers
- ***Image*** object:
  - Defines a two- or three-dimensional region of memory.
  - Image data can only be accessed with read and write functions, i.e. these are opaque data structures. The read functions use a sampler.

Creating and manipulating buffers

- Buffers are declared on the host as type: `cl_mem`

- Arrays in host memory hold your original host-side data:

```
float h_a[LENGTH], h_b[LENGTH];
```

- Create the buffer (d_a), assign sizeof(float)*count bytes from "h_a" to the buffer and copy it into device memory:

```
cl_mem d_a = clCreateBuffer(context,
    CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
     sizeof(float)*count, h_a, NULL);
```

Conventions for naming buffers

- It can get confusing about whether a host variable is just a regular C array or an OpenCL buffer

- A useful convention is to prefix the names of your regular **h**ost C arrays with "**h_**" and your OpenCL buffers which will live on the **d**evice with "**d_**"

Creating and manipulating buffers

- Other common memory flags include:

  `CL_MEM_WRITE_ONLY, CL_MEM_READ_WRITE`

- These are from the point of view of the **device**

- Submit command to copy the buffer back to host memory at "h_c":
  - CL_TRUE = blocking, CL_FALSE = non-blocking

```
clEnqueueReadBuffer(queue, d_c, CL_TRUE,
        sizeof(float)*count, h_c,
        NULL, NULL, NULL);
```

4. Define the kernel

- Create kernel object from the kernel function "vadd":

```
kernel = clCreateKernel(program, "vadd", &err);
```

- Attach arguments of the kernel function "vadd" to memory objects:

```
err  = clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_a);
err |= clSetKernelArg(kernel, 1, sizeof(cl_mem), &d_b);
err |= clSetKernelArg(kernel, 2, sizeof(cl_mem), &d_c);
err |= clSetKernelArg(kernel, 3, sizeof(unsigned int),&count);
```

5. Enqueue commands

- Write Buffers from host into global memory (as non-blocking operations):

```
err = clEnqueueWriteBuffer(commands, d_a, CL_FALSE,
          0, sizeof(float)*count, h_a, 0, NULL, NULL);
err = clEnqueueWriteBuffer(commands, d_b, CL_FALSE,
          0, sizeof(float)*count, h_b, 0, NULL, NULL);
```

- Enqueue the kernel for execution (note: in-order so OK):

```
err = clEnqueueNDRangeKernel(commands, kernel, 1,
                  NULL, &global, &local, 0, NULL, NULL);
```

5. Enqueue commands

- Read back result (as a blocking operation). We have an in-order queue which assures the previous commands are completed before the read can begin.

```
err = clEnqueueReadBuffer(commands, d_c, CL_TRUE,
                          sizeof(float)*count, h_c, 0, NULL, NULL);
```

Vector Addition – Host Program

Define platform and queues

```
// create the OpenCL context on a GPU device
cl_context context = clCreateContextFromType(0,
                     CL_DEVICE_TYPE_GPU, NULL, NULL, NULL);
// get the list of GPU devices associated with context
clGetContextInfo(context, CL_CONTEXT_DEVICES, 0, NULL, &cb);
cl_device_id[] devices = malloc(cb);
clGetContextInfo(context,CL_CONTEXT_DEVICES,cb,devices,NULL);
// create a command-queue
cmd_queue = clCreateCommandQueue(context,devices[0],0,NULL);
```

Define memory objects

```
// allocate the buffer memory objects
memobjs[0] = clCreateBuffer(context, CL_MEM_READ_ONLY |
       CL_MEM_COPY_HOST_PTR, sizeof(cl_float)*n, srcA, NULL);
memobjs[1] = clCreateBuffer(context, CL_MEM_READ_ONLY |
       CL_MEM_COPY_HOST_PTR, sizeof(cl_float)*n, srcb, NULL);
memobjs[2] = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
                     sizeof(cl_float)*n, NULL, NULL);
```

Create the program

```
// create the program
program = clCreateProgramWithSource(context, 1,
                     &program_source, NULL, NULL);
```

Build the program

```
// build the program
err = clBuildProgram(program, 0, NULL,NULL,NULL,NULL);
```

Create and setup kernel

```
// create the kernel
kernel = clCreateKernel(program, "vec_add", NULL);
// set the args values
err  = clSetKernelArg(kernel, 0, (void *) &memobjs[0],
                     sizeof(cl_mem));
err |= clSetKernelArg(kernel, 1, (void *) &memobjs[1],
                     sizeof(cl_mem));
err |= clSetKernelArg(kernel, 2, (void *) &memobjs[2],
                     sizeof(cl_mem));
// set work-item dimensions
global_work_size[0] = n;
```

Execute the kernel

```
// execute kernel
err = clEnqueueNDRangeKernel(cmd_queue, kernel, 1, NULL,
                     global_work_size, NULL,0,NULL,NULL);
```

Read results on the host

```
// read output array
err = clEnqueueReadBuffer(cmd_queue, memobjs[2],
                     CL_TRUE, 0, n*sizeof(cl_float), dst,
                     0, NULL, NULL);
```

OpenCL versus CUDA

CUDA:

- Scalar Core
- Streaming Multiprocssr
- Warp
- PTX

OpenCL:

Stream Core

Compute Unit

Wavefront

Intermediate Language

OpenCL versus CUDA

CUDA:

- Host Memory
- Global/Device Memory
- Local Memory
- Constant Memory
- Shared Memory
- Registers

OpenCL:

Host Memory

Global Memory

Global Memory

Constant Memory

Local Memory

Private Memory

OpenCL versus CUDA

CUDA:

- Grid
- Block
- Thread
- Thread ID
- Block Index
- Thread Index

OpenCL:

NDRange

Work group

Work item

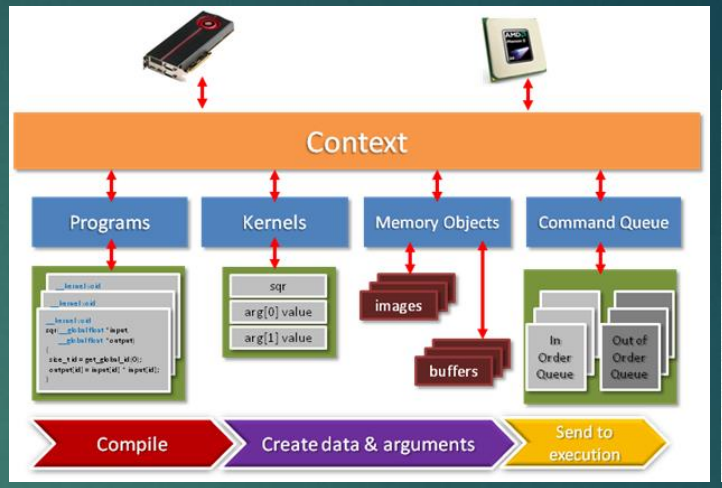
Global ID

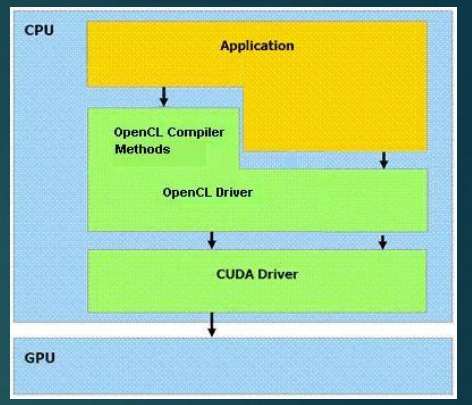
Block ID

Local ID

Conclusion

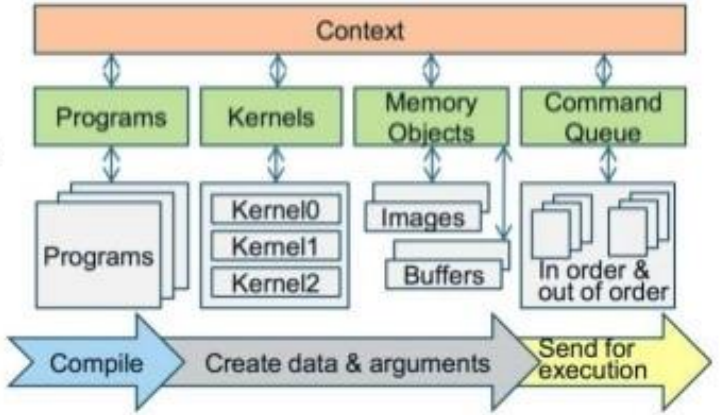
- OpenCL Executing Model
  - Work Items>Work Groups => NDRange
- OpenCL Context & **Command Queue**
- Hierarchical Memory
  - Private>Local>Constant>Global>Host
- **Kernels build on Runtime**
- OpenCL APIs
  - Platform/Context
  - **Kernels/Programs Build**
  - Memory Objects
  - Kernel Functions Execution
  - **Command Queue**
- OpenCL versus CUDA





OpenCL:Execution of Programs - Steps

1. Query host for OpenCL devices.
2. Create a context to associate OpenCL devices.
3. Create programs for execution on one or more associated devices.
4. From the programs, select kernels to execute.
5. Create memory objects accessible from the host and/or the device.
6. Copy memory data to the device as needed.
7. Provide kernels to the command queue for execution.
8. Copy results from the device to the host.



END

QUESTIONS & ANSWERS